Risk as an Effect: Static Enforcement of Quantitative Policies

SEPEHR AMIR, University of the Pacific, USA

Modern software systems must reason not only about functional correctness, but also about the consequences of execution. Static quantitative analyses can bound properties such as resource consumption or domain-specific metrics, but risk-sensitive applications introduce an additional distinction between the behavior that gives rise to a hazard and the consequence assigned to that hazard. We present a semantic framework and type-and-effect system for statically enforcing quantitative risk policies. The framework factors reasoning about risk into execution behavior, hazard recognition, quantitative loss, and policy bounds. Hazards characterize undesirable conditions induced by execution, while a domain-specific loss model assigns their quantitative consequences. A compositional type-and-effect analysis conservatively approximates these quantities and enables programs to be certified against quantitative policies without enumerating executions. We establish soundness for both hazard and risk approximation and extend the framework in three directions. Prefix-based semantics supports potentially non-terminating computations; symbolic risk expressions permit policy checking under explicit assumptions on program parameters; and an abstract history domain supports hazards whose occurrence depends on ordering, repetition, or prior events rather than on individual operations alone. We implement a prototype checker for a subset of the Workflow Description Language (WDL), demonstrating base quantitative analysis, history-sensitive hazard detection, symbolic bounded repetition, constraint-based policy checking, and diagnostic reporting. The resulting framework provides a foundation for consequence-aware static analysis in which behavioral abstraction and domain-specific valuation remain explicitly separated.

Additional Key Words and Phrases: Type-and-effect systems; Static analysis; Quantitative risk

1 Introduction

Modern software systems increasingly operate in environments where functional correctness alone is insufficient. In domains such as safety-critical systems, cybersecurity, cloud infrastructure, and data-driven services, system behavior may incur consequences even when execution remains functionally correct. Such consequences may include information exposure, availability degradation, financial loss, safety impact, or operational cost [9, 29]. In these settings, it is useful to reason not only about whether an undesirable event is possible, but also about the magnitude of the consequences that executions may incur.

Static program analyses already provide powerful forms of quantitative reasoning. Resource-aware type systems infer bounds on quantities such as time, memory, or energy consumption [26]; quantitative information-flow analyses measure information leakage; and quantitative verification reasons about costs, probabilities, and rewards over behavioral models. Likewise, effect systems provide compositional abstractions of program behavior, including systems in which effects record ordering or execution history. These techniques demonstrate that quantitative and history-sensitive properties can be analyzed compositionally.

Risk-sensitive reasoning introduces a somewhat different separation of concerns. The behavior that makes an execution hazardous need not itself be the quantity against which policy is stated. For example, a deployment may first create public network exposure and later attach a customer database. Neither operation need be independently unacceptable, yet their combination may constitute a data-exposure hazard. The quantitative consequence assigned to that hazard—for example, security impact, availability impact, or financial loss—is a separate domain-specific judgment. A different organization may assign a different consequence to the same execution pattern without changing the underlying program behavior.

We therefore view quantitative risk through four related but distinct layers:

$$\text{execution behavior} \longrightarrow \text{hazards} \longrightarrow \text{loss} \longrightarrow \text{policy}.$$

Author's Contact Information: Sepehr Amir, University of the Pacific, Stockton, California, USA, samir1@pacific.edu.

Hazards identify undesirable conditions induced by execution. A loss model interprets those hazards quantitatively in a domain-specific risk space, and a policy specifies which resulting bounds are acceptable. This factorization is deliberate: the static analysis should approximate program behavior and hazardous conditions without baking one particular quantitative valuation into the semantics of those events.

Runtime monitoring provides one natural way to implement such reasoning: execution traces can be observed, hazardous patterns recognized, and quantitative consequences evaluated as execution proceeds [4, 36]. Static enforcement offers a complementary guarantee. It can conservatively approximate the hazardous behavior of all modeled executions before deployment and reuse that result across executions. The challenge is therefore not simply to attach numbers to program operations, but to connect a semantic account of hazardous behavior to a compositional static analysis and to quantitative policy checking.

Type-and-effect systems provide a convenient realization of this idea. They augment ordinary typing judgments with compositional summaries of execution behavior [20, 33, 35, 44]. Quantitative resource analyses similarly demonstrate that numerical bounds can be propagated compositionally through program structure [26]. Our goal is not to replace these techniques or to claim that quantitative effects, symbolic bounds, or ordered effects are themselves new. Rather, we use a type-and-effect system to realize a semantic interface in which hazardous behavior and its quantitative interpretation remain explicitly separated.

Building on this view, we introduce a *risk-aware type-and-effect system* for static enforcement of quantitative policies. In the core system, judgments have the form

$$\Gamma \vdash e : \tau \ \triangleright \ (H, r),$$

where H conservatively approximates hazards that may arise during execution and r bounds worst-case quantitative risk. The two components play related but distinct roles. The hazard component records behavioral information that can be interpreted by a domain-specific loss model, while the risk component provides a control-flow-sensitive quantitative bound suitable for policy checking. The semantic development relates both components to execution behavior and establishes separate soundness guarantees for hazard approximation and quantitative risk.

This separation becomes especially important when hazards do not correspond to individual operations. The core analysis intentionally begins with a flat hazard abstraction suitable for memoryless hazards. We then generalize this abstraction to an abstract history domain Q. A history summary records information such as ordering, repetition, or abstract protocol state; a hazard interpretation

$$\eta : Q \to \mathcal{P}(W)$$

determines which hazards follow from that history; and a loss model

$$L : \mathcal{P}(W) \to X$$

assigns their quantitative consequences. Thus, quantitative loss may arise from a combination of actions even when no individual action carries that loss locally.

The framework also accommodates two additional concerns that arise naturally in workflow-oriented applications. First, potentially non-terminating executions are interpreted through finite execution prefixes, so that unbounded accumulation can be represented explicitly. Bounded repetition, in contrast, admits finite or symbolic summaries. Second, symbolic quantitative bounds may be checked relative to assumptions on deployment or input parameters. This allows a single compositional summary to certify a family of executions under explicit environmental constraints.

The resulting framework supports static certification of quantitative risk policies before execution. Programs whose inferred bounds exceed an acceptable policy can be rejected statically, while

runtime mechanisms remain complementary for assumptions or behaviors outside the static model. We instantiate the framework in a prototype checker for a subset of the Workflow Description Language (WDL) [38]. WDL is a useful setting because workflow structure makes sequencing, branching, and bounded repetition explicit, while domain-specific hazard, loss, and policy information may be provided independently of the workflow implementation. The prototype lowers WDL workflows to a core intermediate representation, obtains primitive and history annotations from an external specification, and checks inferred summaries against quantitative policies.

Contributions. This paper makes the following contributions.

(1) We define a semantic framework for consequence-aware static reasoning that separates execution behavior, hazard recognition, quantitative loss, and policy certification. This factorization makes the interpretation of hazardous behavior explicit without identifying behavioral effects with a fixed quantitative valuation.
(2) We provide a sound compositional type-and-effect realization of this framework. The analysis derives conservative behavioral summaries and quantitative risk bounds, with separate soundness guarantees. We establish conditions under which behavioral summaries can be reused under alternative loss models and, for history-sensitive analyses, alternative hazard interpretations.
(3) We demonstrate how the architecture accommodates several forms of quantitative reasoning using established analysis mechanisms. These include abstract histories for recognizing compound and temporal hazards, finite-prefix semantics and fixed-point bounds for iteration, and symbolic quantitative summaries certified under explicit environmental assumptions.
(4) We implement a prototype checker for a subset of WDL, demonstrating the integration of behavioral annotations, history-sensitive hazard recognition, symbolic bounded repetition, quantitative policy checking, and diagnostic reporting.

1.1 Motivating Example

Consider a deployment pipeline for a cloud service. Before applying an infrastructure change, the pipeline checks whether the change has been approved, provisions the service, configures network exposure, attaches a database, and enables audit logging. Such workflows are common in infrastructure-as-code and continuous-deployment settings, where unsafe deployments may expose data, disrupt availability, or incur operational cost [39].

Figure 1 shows a simplified deployment workflow and the risk assigned to each primitive operation. These risks are supplied by a policy, security review, service catalog, auxiliary analysis, or other domain-specific source, rather than inferred from cloud-operation implementations. The type-and-effect system composes the per-operation summaries through the workflow and checks the resulting risk against the deployment policy.

Let risk be a three-dimensional quantity whose components represent security exposure, availability risk, and operational cost. The numbers in Figure 1 are illustrative. For instance, public ingress contributes to security exposure, availability risk, and operational cost, customer-database attachment contributes most strongly to availability risk and operational cost, and provisioning contributes most strongly to operational cost.

The workflow risk is computed by composing the primitive risk summaries along program structure. Sequential operations accumulate risk, while conditionals use a conservative branch-wise bound. Applying this compositional calculation to the workflow gives risk $(4, 4, 10)$ for the approved branch and $(0, 0, 2)$ for the rejection branch. Taking the conservative bound across the outer conditional yields an overall risk bound of $r = (4, 4, 10)$. For a deployment policy $\rho = (5, 5, 12)$,

Workflow

```
if (approved(change)) then {
  provision_service(change);

  if (public_endpoint(change)) then
    create_public_ingress(service)
  else
    create_private_ingress(service);

  if (uses_customer_db(change)) then
    attach_customer_db(service)
  else
    attach_test_db(service);

  enable_audit_logging(service)
} else {
  reject_change(change)
}
```

Primitive risks

`approved`	$(0, 0, 1)$
`provision_service`	$(1, 1, 3)$
`public_endpoint`	$(0, 0, 1)$
`uses_customer_db`	$(0, 0, 1)$
`create_public_ingress`	$(2, 1, 1)$
`create_private_ingress`	$(0, 1, 1)$
`attach_customer_db`	$(1, 2, 2)$
`attach_test_db`	$(0, 0, 1)$
`enable_audit_logging`	$(0, 0, 1)$
`reject_change`	$(0, 0, 1)$

Fig. 1. A deployment workflow and illustrative primitive risk summaries.

each component of r is within the corresponding component of ρ, so this coarse multi-dimensional bound alone would allow the deployment.

However, some risks arise from combinations of operations rather than independent per-operation costs. Creating a public ingress and attaching a customer database are each locally acceptable under the primitive summaries, but together they may expose customer data through a public-facing service. Suppose the policy assigns a history-sensitive loss of $(8, 0, 0)$ to executions in which both operations occur. Taking a conservative bound across the approved-branch risk and this history-sensitive loss yields $(8, 4, 10)$, exceeding the policy bound ρ in the security dimension. The deployment is therefore rejected because of the combination of public exposure and customer-data attachment.

This example illustrates the distinction at the center of the framework. Primitive operations may carry local quantitative summaries, but the combination of public ingress and customer-data attachment demonstrates a different form of risk. The two operations first contribute behavioral information to the execution history; their composition entails the compound hazard w_{pubdb}; and only then does the loss model assign the quantitative consequence $(8, 0, 0)$. Thus, the consequence is not required to be attached locally to either operation. The same hazardous execution pattern may also be interpreted under a different loss model without changing the workflow or its behavioral abstraction. Sections 3 and 6 formalize the memoryless and history-sensitive cases, respectively.

Paper Outline. Section 2 establishes the semantic foundations, and Section 3 develops the core type-and-effect system. We progressively extend this foundation to non-terminating executions and iteration (Section 4) and symbolic, assumption-guarded quantitative policies (Section 5). Section 6 then generalizes the behavioral abstraction to execution histories, enabling recognition of compound and temporal hazards. Section 7 presents the prototype. Sections 8 and 9 discuss the future and related work, respectively, and Section 10 concludes.

2 Semantic Model of Risk and Sound Abstraction

We formalize quantitative risk through four distinct layers: *execution behavior*, *hazard recognition*, *quantitative loss*, and *policy certification*. Execution semantics determines which hazardous events may arise, a loss model assigns quantitative consequences to those hazards, and a policy specifies

which consequences are acceptable. Separating these layers allows a behavioral abstraction to be reused independently of a particular quantitative interpretation.

We first define hazards and risk semantically over program executions. We then give a language-independent interface for static analyses that conservatively approximate these quantities. Section 3 instantiates this interface with a type-and-effect system.

Detailed proofs of the results are given in Appendix A.

2.1 Execution and Hazard Semantics

We initially consider terminating executions. An execution trace is a finite sequence $\pi = e_0 \xrightarrow{H_0} e_1 \xrightarrow{H_1} \cdots \xrightarrow{H_{n-1}} e_n$, representing an execution from an initial expression to a terminal configuration. We write $\mathsf{Traces}(e)$ for the set of terminating traces starting from e.

Fix a set W of atomic hazards. Each evaluation step may emit a set of hazards, written $e \xrightarrow{H} e'$, where $H \subseteq W$. The transition labels provide the semantic interface between program execution and hazard recognition.

In the core model, hazards are associated directly with evaluation steps. Section 6 generalizes this interface so that hazards may instead be recognized from abstract execution histories, including combinations of events that are not individually hazardous.

The hazards occurring along a trace π are $\mathsf{hazards}(\pi) = \bigcup_{i<n} H_i$. The semantic hazard set of an expression collects all hazards that may arise over its terminating executions: $\Psi(e) = \bigcup_{\pi \in \mathsf{Traces}(e)} \mathsf{hazards}(\pi)$.

We also consider finite execution segments, which need not terminate. If two segments π_1 and π_2 are composable, we write $\pi_1 \cdot \pi_2$ for their concatenation.

Lemma 2.1 (Hazard Composition for Trace Concatenation). *For any composable execution segments π_1 and π_2,* $\mathsf{hazards}(\pi_1 \cdot \pi_2) = \mathsf{hazards}(\pi_1) \cup \mathsf{hazards}(\pi_2)$.

Proof. Immediate from the definition of hazard accumulation. Details are given in Appendix A.1.

The restriction to terminating traces is sufficient for the initial core semantics. Section 4 extends these definitions to finite prefixes of potentially non-terminating executions.

2.2 Loss and Quantitative Risk

We model quantitative consequences in the domain $X = \mathbb{R}^d_{\geq 0}$, ordered componentwise by $\preceq$. Thus, $r \preceq r'$ iff $r_i \leq r'_i$ for every component i. We write $\mathbf{0} \in X$ for the zero vector, which is the least element of X.

We use two operations to combine quantitative bounds. Sequential composition is represented by $\oplus : X \times X \to X$, while $\sqcup : X \times X \to X$ combines bounds arising from alternative control-flow paths.

We assume that $\oplus$ is associative, monotone in both arguments, and has $\mathbf{0}$ as its identity. Thus, for all $r, r_1, r_2 \in X$, $\mathbf{0} \oplus r = r \oplus \mathbf{0} = r$. Moreover, if $r_1 \preceq r'_1$ and $r_2 \preceq r'_2$, then $r_1 \oplus r_2 \preceq r'_1 \oplus r'_2$.

The operator $\sqcup$ is assumed to be monotone and to upper-bound both operands: $r_1 \preceq r_1 \sqcup r_2$ and $r_2 \preceq r_1 \sqcup r_2$.

For the concrete vector instantiation used in our examples, $\oplus$ is componentwise addition and $\sqcup$ is componentwise maximum. The latter is the least upper bound under the componentwise order.

Since $\mathbf{0}$ is least and $\oplus$ is monotone, these assumptions also imply $r \preceq r \oplus r'$ for all $r, r' \in X$. This property will be used when reasoning about execution prefixes that stop before a computation has completed.

Loss models. A *loss model* is a function $L : \mathcal{P}(W) \to X$ assigning quantitative consequences to sets of hazards.

We require L to be monotone: $H \subseteq H'$ implies $L(H) \preceq L(H')$. We also assume $L(\emptyset) = \mathbf{0}$.

The dependence of semantic risk on the loss model is made explicit in the notation. For a trace π, define $R_L(\pi) = L(\mathsf{hazards}(\pi))$.

The semantic risk of an expression is the worst-case loss over its terminating executions: $R_L(e) = \bigsqcup_{\pi \in \mathsf{Traces}(e)} R_L(\pi)$.

Here, $\bigsqcup$ denotes the least upper bound of the corresponding set of risk vectors. We initially consider expressions for which this bound exists in X, as is the case for the finite core language under finite primitive annotations and finitely many possible hazard sets. Section 4 extends the quantitative domain with ∞ to accommodate unbounded risk.

Thus, execution semantics determines which hazards arise, while L determines how those hazards are valued quantitatively. Different loss models may assign different consequences to the same program behavior without changing its execution or hazard semantics.

Compatibility with sequential composition. For compositional static reasoning, we additionally require the loss model to be compatible with the quantitative sequential-composition operator.

Assumption 2.2 (Sequential Compositionality of Loss). *For all $H_1, H_2 \subseteq W$, $L(H_1 \cup H_2) \preceq L(H_1) \oplus L(H_2)$.*

This condition states that the loss associated with the union of two hazard sets must be conservatively bounded by the sequential composition of their losses.

The assumption does not prescribe a particular loss function. Rather, it specifies the compatibility required between a chosen loss model and the quantitative composition operator used by the analysis.

Importantly, this condition limits which interactions can be represented solely through the flat hazard-set semantics. A consequence arising specifically from a combination of operations may not be bounded by composing their individual losses. Such consequences can instead be represented as additional hazards recognized from composed execution histories, as developed in Section 6.

2.3 Static Abstraction and Soundness Criteria

A static analysis assigns an expression e an abstract summary (H, r), where $H \subseteq W$ conservatively approximates the hazards that may arise during execution, and $r \in X$ bounds semantic risk under a chosen loss model.

The two components provide different guarantees. The hazard component H is a behavioral abstraction that can be interpreted independently through a loss model. The quantitative component r is a direct bound that may exploit control-flow structure during analysis.

Definition 2.3 (Hazard Soundness). *A static analysis assigning hazard summary $H \subseteq W$ to e is* hazard sound *if $\Psi(e) \subseteq H$.*

Definition 2.4 (Risk Soundness). *For a loss model L, a static analysis assigning risk bound $r \in X$ to e is* risk sound with respect to L *if $R_L(e) \preceq r$.*

Hazard soundness alone already induces a quantitative guarantee under every monotone interpretation of the hazards.

Theorem 2.5 (Loss-Parametric Revaluation). *Suppose a static hazard summary H for e is hazard sound, i.e., $\Psi(e) \subseteq H$. Then, for every monotone loss model $L : \mathcal{P}(W) \to X$ for which $R_L(e)$ is defined, $R_L(e) \preceq L(H)$.*

Proof. For every $\pi \in \mathsf{Traces}(e)$, $\mathsf{hazards}(\pi) \subseteq \Psi(e) \subseteq H$. Monotonicity of L gives $R_L(\pi) \preceq L(H)$. Taking the least upper bound over all terminating traces establishes the result. Details are given in Appendix A.2.

Theorem 2.5 formalizes the reusable role of the hazard abstraction. Once H has been computed soundly, the same behavioral summary can be interpreted under a different monotone loss model without recomputing the hazard analysis, provided that the underlying execution-to-hazard semantics remains unchanged.

Unlike direct risk soundness, this result does not require Assumption 2.2. It relies only on hazard soundness and monotonicity of the loss interpretation.

Precision of the two summaries. The revalued bound $L(H)$ may be less precise than the directly computed quantitative bound r.

For example, suppose one branch of a conditional may emit w_1 and another may emit w_2. The static hazard summary conservatively contains $H = \{w_1, w_2\}$, even though the two hazards cannot occur together in one execution.

Revaluation therefore considers $L(\{w_1, w_2\})$, whereas the direct quantitative analysis may preserve branch exclusivity through $r_1 \sqcup r_2$.

The two bounds are not generally ordered relative to each other. Both may conservatively approximate the same semantic risk, but their precision depends on the loss model, primitive annotations, and abstractions used by the analysis.

Consequently, the hazard component provides a reusable behavioral summary, while the direct quantitative component can provide a more precise certificate for a particular analysis instance.

2.4 Policy Satisfaction and Static Certification

A quantitative policy is a bound $\rho \in X$. We distinguish semantic satisfaction of a policy from successful static certification.

Definition 2.6 (Semantic Policy Satisfaction). *An expression e satisfies policy ρ under loss model L, written $e \models_L \rho$, if $R_L(e) \preceq \rho$.*

A static analysis certifies the policy when its inferred risk bound lies below the policy threshold.

Definition 2.7 (Static Policy Certification). *A risk-sound static summary (H, r)* certifies *e against policy ρ when $r \preceq \rho$.*

Risk soundness connects static certification with semantic policy satisfaction.

Corollary 2.8 (Sound Static Certification). *If the analysis is risk sound with respect to L and $r \preceq \rho$, then $e \models_L \rho$.*

The result follows immediately from $R_L(e) \preceq r \preceq \rho$.

The hazard component provides an alternative, potentially more conservative, form of certification.

Corollary 2.9 (Certification by Revaluation). *If H is hazard sound and $L(H) \preceq \rho$, then $e \models_L \rho$.*

This follows from Theorem 2.5 and transitivity.

Consequently, an analysis may use the directly inferred quantitative bound for precision while retaining the behavioral summary for revaluation under alternative loss models.

A failed static certification does not necessarily imply semantic policy violation. Since the analysis is conservative, a bound exceeding ρ may reflect approximation rather than an actual execution whose loss exceeds the policy.

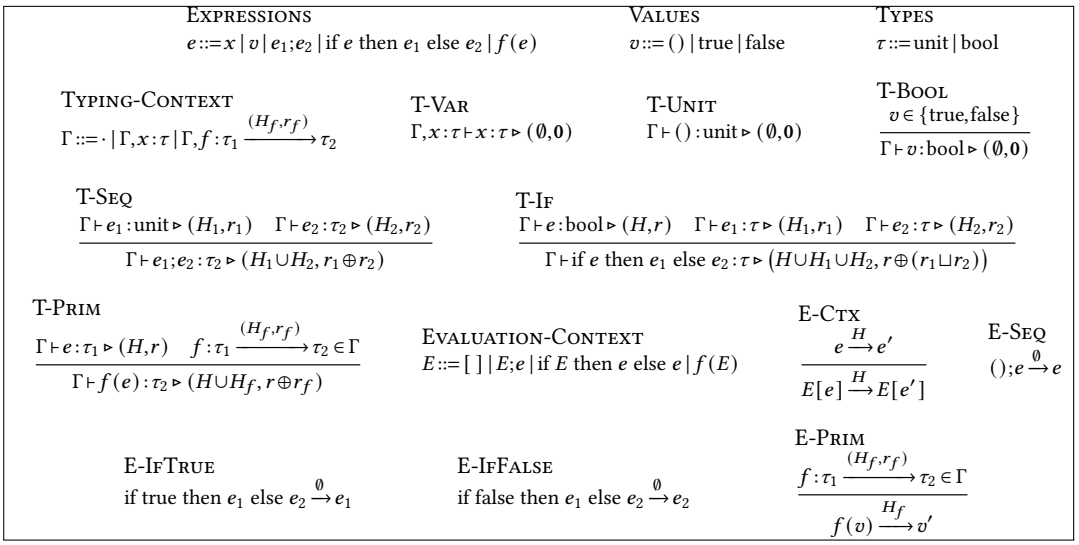


Fig. 2. Core language: syntax, type-and-effect rules, evaluation contexts, and hazard-labeled operational semantics.

3 Core Type-and-Effect Analysis

We instantiate the semantic interface of Section 2 with a syntax-directed type-and-effect analysis. The analysis computes judgments of the form $\Gamma \vdash e : \tau \triangleright (H, r)$, where H conservatively approximates the hazards associated with e, and r is a direct quantitative bound on its semantic risk.

The two components have complementary purposes. The hazard summary H is independent of the chosen quantitative loss model and can be revalued without recomputing the behavioral abstraction. The bound r, in contrast, is computed using the quantitative annotations of the current analysis instance and may need to be recomputed when those annotations or their loss interpretation change. Because r is propagated through the control-flow structure of the program, it may also be more precise than evaluating the aggregate hazard summary directly.

We present the core language, typing rules, and operational semantics, then establish hazard soundness and direct quantitative risk soundness. The latter is relative to the loss model and the soundness of the externally supplied primitive annotations.

3.1 Syntax and Types

We consider a small call-by-value expression language. Its syntax, values, types, typing rules, and operational semantics are shown in Figure 2.

Here, f ranges over a fixed set of primitive operations, such as `approved`, `reject_change`, and `create_public_ingress`. Primitive operations provide the interface between the core language and domain-specific behavior.

Typing judgments have the form $\Gamma \vdash e : \tau \triangleright (H, r)$, where Γ is the typing context, $H \subseteq W$ is a conservative hazard summary, and $r \in X$ is a direct quantitative risk bound.

The typing rules are syntax directed. Variables and values contribute neither hazards nor risk. Sequencing combines hazards by union and quantitative bounds by $\oplus$, yielding $(H_1 \cup H_2, r_1 \oplus r_2)$.

For a conditional, the hazard component contains hazards from the guard and both branches: $H \cup H_1 \cup H_2$. The quantitative component instead uses $r \oplus (r_1 \sqcup r_2)$, preserving the distinction between mutually exclusive branches.

This distinction is significant. Applying a loss model to the union of hazards from both branches may conservatively combine consequences that cannot occur together in one concrete execution. The direct quantitative analysis instead joins branch-specific bounds.

For example, under an additive loss model, if the two branches produce distinct hazards of loss $(2, 0, 0)$ each, then their combined hazard summary has loss $(4, 0, 0)$, while the direct branch join yields $(2, 0, 0)$, assuming the guard contributes zero risk. Both are sound bounds, but the direct analysis retains the exclusivity of the branches.

Primitive application combines the argument's summary with the latent annotation (H_f, r_f) of the primitive. The hazard annotation describes its behavioral effect, while the quantitative annotation provides a conservative bound for the current loss interpretation.

Primitive Annotations and Trust Boundary. Primitive operations are modeled by annotated function symbols $f : \tau_1 \xrightarrow{(H_f, r_f)} \tau_2$. The set H_f identifies the hazards emitted by executing the primitive, while r_f supplies a quantitative bound on their consequences.

For example, the deployment environment from Section 1.1 may contain `create_public_ingress` : unit $\xrightarrow{(\{w_{\text{pub}}\},(2,1,1))}$ unit and `attach_customer_db` : unit $\xrightarrow{(\{w_{\text{db}}\},(1,2,2))}$ unit.

Here, w_{pub} denotes public network exposure and w_{db} denotes attachment of a customer database.

Primitive annotations are externally supplied summaries rather than quantities inferred by the core calculus from primitive implementations. They may originate from policy, measurement, certification, security review, auxiliary analysis, service catalogs, or other domain-specific sources.

For a fixed loss model L, direct quantitative soundness requires every primitive annotation to conservatively cover the loss associated with its emitted hazards.

Assumption 3.1 (Primitive Quantitative Soundness). *For every primitive annotation* $f : \tau_1 \xrightarrow{(H_f, r_f)} \tau_2$, *we require* $L(H_f) \preceq r_f$.

This assumption is part of the trust boundary of the analysis. The type system establishes soundness relative to the supplied annotations, but does not independently establish their empirical accuracy or adequacy for a particular deployment environment.

The distinction between H_f and r_f is also relevant when the consequence model changes. A behavioral annotation H_f may remain valid under a new loss model L', even when the corresponding quantitative annotation r_f is no longer sound for that model.

In such cases, direct quantitative analysis requires updated annotations or recomputation. Alternatively, a previously inferred hazard summary may be revalued directly through Theorem 2.5.

3.2 Operational Semantics and Hazard Generation

The operational semantics instantiates the execution-to-hazard layer of Section 2. Evaluation contexts enforce a standard call-by-value evaluation order. Reduction steps emit hazard labels but do not directly carry quantitative losses.

Rule E-Ctx propagates hazard labels through evaluation contexts. Pure control-flow reductions, including E-Seq, E-IfTrue, and E-IfFalse, emit no hazards.

Primitive execution is the semantic boundary at which domain-specific events enter the core calculus. Rule E-Prim emits the primitive hazard annotation H_f. The result v' is determined by the primitive's operational behavior and is assumed to have the declared result type τ_2 whenever the

argument value has type τ_1. Primitive behavior may be nondeterministic; the soundness results apply to all executions admitted by the operational semantics.

Importantly, the reduction relation depends on the primitive's behavioral annotation H_f, but not on its quantitative annotation r_f or the loss model L. Quantitative interpretation is introduced separately by the semantic loss model and the type-and-effect analysis.

Thus, the operational semantics realizes the execution–hazard–loss separation introduced in Section 2.

Primitive operations are the only direct source of hazards in the core calculus. This gives a simple interface for modeling effectful operations, including I/O, cloud operations, service interactions, and workflow tasks.

Section 6 replaces the flat behavioral abstraction with abstract histories. In that extension, primitive execution contributes history information, and hazards can be recognized from combinations of events rather than being assigned only to individual primitive steps.

The following lemma records the compositional structure of semantic hazards in the core calculus.

Lemma 3.2 (Compositionality of Hazards). *For all expressions e, e_1, e_2:*

(1) $\Psi(e_1; e_2) \subseteq \Psi(e_1) \cup \Psi(e_2)$;
(2) $\Psi(\mathit{if}\, e\, \mathit{then}\, e_1\, \mathit{else}\, e_2) \subseteq \Psi(e) \cup \Psi(e_1) \cup \Psi(e_2)$;
(3) $\Psi(f(e)) \subseteq \Psi(e) \cup H_f$.

Proof. By case analysis on terminating executions, using Lemma 2.1. Details are given in Appendix A.3.

3.3 Soundness and Revaluation

We establish three properties of the core analysis. First, the inferred hazard component soundly approximates hazardous execution behavior. Second, the behavioral summary can be revalued under alternative monotone loss models without recomputing the hazard analysis. Third, the directly inferred risk component soundly bounds semantic risk for the chosen loss model and primitive quantitative annotations.

The results are stated for the terminating execution semantics of Section 2. Section 4 strengthens them to finite prefixes, including those of potentially non-terminating executions.

Theorem 3.3 (Hazard Soundness of the Type System). *If $\Gamma \vdash e : \tau \triangleright (H, r)$, then $\Psi(e) \subseteq H$.*

Proof. By induction on the typing derivation, using Lemma 3.2. Details are given in Appendix A.4.

Hazard soundness immediately yields a quantitative guarantee through Theorem 2.5.

Corollary 3.4 (Revaluation of Inferred Hazard Summaries). *Suppose $\Gamma \vdash e : \tau \triangleright (H, r)$. Then, for every monotone loss model $L' : \mathcal{P}(W) \to X$, $R_{L'}(e) \preceq L'(H)$.*

Proof. By Theorem 3.3, $\Psi(e) \subseteq H$. The result follows from Theorem 2.5.

The corollary formalizes the reusable role of the hazard component. Changing the quantitative interpretation of hazards does not invalidate H, provided the execution-to-hazard semantics is unchanged.

Consequently, the same behavioral summary can support different consequence models and policy thresholds without repeating the hazard analysis.

The bound $L'(H)$ may be less precise than a direct quantitative bound computed under L'. This occurs because H collects hazards across all possible executions, potentially combining hazards from mutually exclusive branches.

The direct analysis can avoid this particular imprecision by propagating quantitative bounds separately through each branch before joining them.

This advantage is conditional on the availability of sound primitive annotations for the chosen loss model and on the sequential-compositionality assumption. As noted in Section 2, the two resulting bounds are not generally ordered relative to each other.

Theorem 3.5 (Direct Risk Soundness of the Type System). *Fix a loss model L satisfying Assumption 2.2, and suppose primitive annotations satisfy Assumption 3.1.*
If $\Gamma \vdash e : \tau \triangleright (H, r)$, then $R_L(e) \preceq r$.

Proof. By induction on the typing derivation, using Lemmas 2.1 and 3.2, together with Assumptions 2.2 and 3.1. Details are given in Appendix A.5.

Theorem 3.5 and Corollary 3.4 give two complementary bounds: $R_L(e) \preceq r$ for the direct analysis, and $R_{L'}(e) \preceq L'(H)$ for revaluation under an alternative loss model.

The first is tailored to the current consequence interpretation and can preserve control-flow distinctions. The second retains the behavioral information needed to obtain additional quantitative certificates without recomputing the hazard abstraction.

Corollary 3.6 (Policy Certification by Direct Risk Analysis). *If $\Gamma \vdash e : \tau \triangleright (H, r)$ and $r \preceq \rho$, under the assumptions of Theorem 3.5, then $e \models_L \rho$.*

Proof. Theorem 3.5 gives $R_L(e) \preceq r$. Since $r \preceq \rho$, transitivity yields $R_L(e) \preceq \rho$.

Corollary 3.7 (Policy Certification by Revaluation). *Suppose $\Gamma \vdash e : \tau \triangleright (H, r)$. For any monotone loss model $L' : \mathcal{P}(W) \to X$, if $L'(H) \preceq \rho$, then $e \models_{L'} \rho$.*

Proof. By Corollary 3.4, $R_{L'}(e) \preceq L'(H)$. The result follows from $L'(H) \preceq \rho$.

Thus, a type-and-effect judgment exposes both a directly inferred quantitative bound and a reusable behavioral summary from which additional policy certificates can be obtained.

3.4 Core Derivation for the Motivating Example

We instantiate the core typing rules on the deployment example from Section 1.1. Sequential quantitative composition is componentwise addition, and branch join is componentwise maximum.

The approval check and provisioning operation contribute $(0, 0, 1) \oplus (1, 1, 3) = (1, 1, 4)$.

The network-exposure conditional contributes $(0, 0, 1) \oplus \big((2, 1, 1) \sqcup (0, 1, 1)\big) = (2, 1, 2)$.

Similarly, the database conditional contributes $(0, 0, 1) \oplus \big((1, 2, 2) \sqcup (0, 0, 1)\big) = (1, 2, 3)$.

Including audit logging, the approved branch has direct risk bound $(1, 1, 4) \oplus (2, 1, 2) \oplus (1, 2, 3) \oplus (0, 0, 1) = (4, 4, 10)$.

The rejection branch has bound $(0, 0, 1) \oplus (0, 0, 1) = (0, 0, 2)$. Taking the componentwise maximum across the outer branches gives $(4, 4, 10) \sqcup (0, 0, 2) = (4, 4, 10)$.

Thus, the core system derives $r = (4, 4, 10)$, satisfying the deployment policy $\rho = (5, 5, 12)$.

This derivation illustrates how the direct quantitative component accounts for mutually exclusive control-flow paths by joining branch-specific bounds, rather than treating hazards from different branches as necessarily co-occurring.

However, the derivation accounts only for locally supplied primitive risks and the flat hazard semantics.

It does not yet capture the additional consequence arising specifically from the combination of public ingress and customer-database attachment. That consequence is not assigned locally to either primitive.

Section 6 instead composes their behavioral history, recognizes the resulting compound hazard, and applies the loss model to assign its quantitative consequence.

4 Extension to Non-Terminating Executions

The semantic model of Section 2 is initially defined over terminating traces. We extend it to potentially non-terminating executions by reasoning over finite execution prefixes. This preserves the execution–hazard–loss separation and enables sound static reasoning about iterative computations.

We use standard fixed-point reasoning for unbounded iteration and finite quantitative composition for bounded repetition. Section 5 subsequently generalizes bounded repetition to symbolic iteration measures and assumption-guarded policy certification.

4.1 Prefix-Based Semantics

A finite execution prefix is a sequence $\sigma = e_0 \xrightarrow{H_0} e_1 \xrightarrow{H_1} \cdots \xrightarrow{H_{n-1}} e_n$, representing an initial segment of a terminating or non-terminating execution. We write $\mathsf{Prefixes}(e)$ for the set of all finite prefixes starting from e, including the empty prefix.

Hazards accumulate along prefixes as they do along terminating traces: $\mathsf{hazards}(\sigma) = \bigcup_{i<n} H_i$.

We extend the semantic hazard set to all finite prefixes: $\Psi(e) = \bigcup_{\sigma \in \mathsf{Prefixes}(e)} \mathsf{hazards}(\sigma)$.

Thus, $\Psi(e)$ includes every hazard that may arise after any finite amount of execution, regardless of whether the computation eventually terminates.

For a fixed loss model L, prefix risk is defined by $R_L(\sigma) = L(\mathsf{hazards}(\sigma))$. The semantic risk of an expression is now $R_L(e) = \bigsqcup_{\sigma \in \mathsf{Prefixes}(e)} R_L(\sigma)$.

Consequently, semantic risk is the worst-case loss over all finite execution prefixes. This definition covers both terminating and non-terminating executions.

From this section onward, $\Psi(e)$ and $R_L(e)$ refer to these prefix-based definitions. On terminating programs, they conservatively extend the trace-based semantics of Section 2.

Extended quantitative domain. To accommodate unbounded quantitative bounds, we extend the risk domain to $X_\infty = (\mathbb{R}_{\geq 0} \cup \{\infty\})^d$, ordered componentwise.

The finite domain X embeds naturally into X_∞, whose least element is $\mathbf{0}$ and whose greatest element is $\top = (\infty, \ldots, \infty)$.

We require the quantitative operations to admit monotone extensions to X_∞ that preserve the algebraic properties stated in Section 2. In particular, $\oplus$ remains associative and monotone with identity $\mathbf{0}$, and $\sqcup$ remains a monotone upper-bound operator.

For the concrete vector instantiation used in our examples, $\oplus$ is componentwise addition and $\sqcup$ is componentwise maximum, extended to include ∞ in the standard way.

The semantic loss model L can be viewed as taking values in X_∞ through the inclusion $X \subseteq X_\infty$. We also permit alternative loss models with values directly in X_∞.

Because X_∞ is a complete lattice under componentwise ordering, the supremum defining $R_L(e)$ exists for every set of finite execution prefixes.

The availability of ∞ allows the analysis to express unbounded quantitative summaries. It does not imply that every non-terminating execution necessarily incurs infinite semantic loss.

Prefix soundness of the core analysis. The proofs of Section 3 establish soundness over terminating traces. For iterative programs, we also need to bound executions that stop before a syntactic computation has completed.

The algebraic assumptions introduced in Section 2 support this strengthening. Since $\mathbf{0}$ is least and $\oplus$ is monotone with identity $\mathbf{0}$, we have $r \preceq r \oplus r'$ for all $r, r' \in X_\infty$.

Therefore, a quantitative summary for a complete sequential computation also upper-bounds a prefix that stops before its subsequent components execute, provided the subexpression summaries are themselves prefix sound.

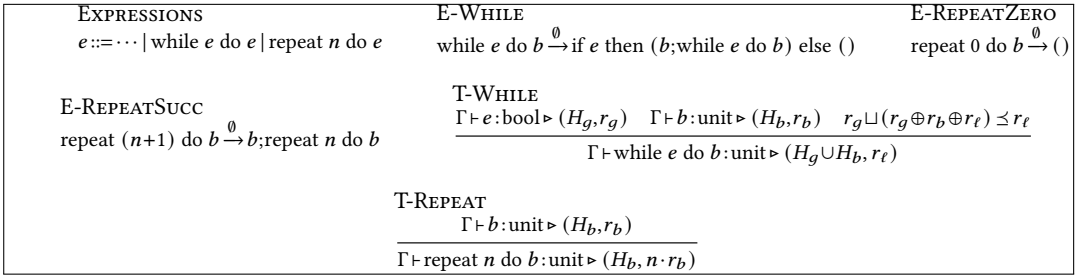


Fig. 3. Extension of the core language with unbounded iteration and bounded repetition.

Lemma 4.1 (Prefix Soundness of Core Constructs). *Suppose* $\Gamma \vdash e : \tau \triangleright (H, r)$ *is derived using the core typing rules. Then every finite execution prefix* σ *of* e *satisfies* $\mathsf{hazards}(\sigma) \subseteq H$ *and* $L(\mathsf{hazards}(\sigma)) \preceq r$*, under Assumptions 2.2 and 3.1.*

Moreover, the core typing rules preserve these properties when their immediate subexpressions are themselves prefix sound.

Proof. By induction on the typing derivation, distinguishing prefixes that remain within a subexpression from those that continue into subsequent computation. Details are given in Appendix A.6.

The second part of the lemma permits the core-rule cases to be reused in the structural soundness argument for the extended language, including expressions containing nested iteration.

The loss-parametric interpretation of hazard summaries also extends to prefix semantics.

Lemma 4.2 (Loss-Parametric Revaluation over Prefixes). *Suppose* H *is a sound prefix-hazard summary for* e*, i.e.,* $\Psi(e) \subseteq H$*. Then, for every monotone loss model* $L' : \mathcal{P}(W) \to X_\infty$*, we have* $R_{L'}(e) \preceq L'(H)$*.*

Proof. For every $\sigma \in \mathsf{Prefixes}(e)$, $\mathsf{hazards}(\sigma) \subseteq \Psi(e) \subseteq H$. Monotonicity of L' gives $R_{L'}(\sigma) \preceq L'(H)$. Taking the supremum over all finite prefixes establishes the result. Details are given in Appendix A.7.

4.2 Iteration

We extend the core language with two constructs: while e do b and repeat n do b.

The first represents potentially unbounded iteration. In this section, $n \in \mathbb{N}$, so bounded repetition executes its body exactly n times. Section 5 generalizes the repetition bound to symbolic parameters.

Figure 3 gives the operational and typing rules.

Unbounded iteration. Rule E-While defines the standard unfolding semantics. The unfolding step emits no hazards; all hazardous behavior arises from evaluation of the guard and body.

Rule T-While assigns the loop a quantitative summary $r_\ell \in X_\infty$ satisfying $r_g \sqcup (r_g \oplus r_b \oplus r_\ell) \preceq r_\ell$.

The first alternative accounts for evaluating the guard and terminating without executing the body. The second accounts for evaluating the guard, executing one body iteration, and returning to the loop.

Equivalently, define the loop transformer $F(x) = r_g \sqcup (r_g \oplus r_b \oplus x)$. The typing premise requires $F(r_\ell) \preceq r_\ell$, so r_ℓ is a pre-fixed point of the transformer.

This is standard fixed-point reasoning: a pre-fixed point conservatively summarizes every finite unfolding of the iterative computation [8].

Because X_∞ contains a greatest element, a conservative pre-fixed point always exists: $\top$ is a pre-fixed point of every transformer mapping X_∞ to itself.

For the concrete vector domain with componentwise addition, a positive guard or body contribution may force the corresponding component of r_t to be infinite when no independent iteration bound is available.

However, this does not necessarily mean that the semantic risk $R_L(e)$ is infinite.

In the flat hazard semantics, hazards are accumulated by set union. Once a particular hazard has occurred, repeated occurrences do not introduce additional copies of the same hazard into the set.

For example, a loop repeatedly executing a primitive with hazard set $\{w\}$ has semantic hazard sets contained in $\{w\}$ for every execution prefix. If $L(\{w\})$ is finite, its semantic risk remains bounded by that finite value, even if the additive loop analysis assigns an infinite bound because it conservatively accounts for arbitrarily many iterations.

Thus, ∞ represents a conservative upper bound produced by the chosen quantitative abstraction. It does not establish that non-termination is intrinsically hazardous or that semantic loss necessarily diverges.

This distinction also motivates the richer behavioral abstractions of Section 6, which can recognize hazards depending on the number or ordering of repeated events.

Bounded repetition. Bounded repetition admits a finite compositional risk summary.

If $\Gamma \vdash b : \text{unit} \triangleright (H_b, r_b)$, then executing b exactly n times has direct quantitative bound $n \cdot r_b$.

For a general quantitative monoid, $n \cdot r_b$ denotes repeated composition: $0 \cdot r_b = \mathbf{0}$ and $(n+1) \cdot r_b = r_b \oplus (n \cdot r_b)$.

In the concrete vector domain with componentwise addition, this coincides with ordinary scalar multiplication.

For example, if $r_b = (1, 1, 2)$, then repeat n do b has risk bound $(n, n, 2n)$.

The hazard component remains H_b, because the flat abstraction records which hazard identities may occur, not their multiplicities.

For $n = 0$, the rule still assigns H_b, although the expression emits no hazards. This is a conservative over-approximation that keeps the typing rule uniform.

A more precise analysis could use an empty hazard summary for zero repetitions. The present rule instead prioritizes simple compositional propagation.

Repetition-sensitive hazards require a richer behavioral abstraction, as developed in Section 6.

Here n is a concrete natural number. The symbolic expression $n \cdot r_b$ motivates Section 5, where repetition counts may be unknown parameters constrained by assumptions.

4.3 Soundness and Revaluation for Iteration

We first record how semantic hazards decompose for the two iterative constructs.

Lemma 4.3 (Unbounded Loop Hazard Decomposition). *For all e and b, $\Psi(\text{while } e \text{ do } b) \subseteq \Psi(e) \cup \Psi(b)$.*

Proof. By induction over finite loop unfoldings. Each unfolding emits no hazards, so all hazards arise from guard or body evaluations. Details are given in Appendix A.8.

Lemma 4.4 (Bounded Repetition Hazard Decomposition). *For all $n \in \mathbb{N}$ and b, $\Psi(\text{repeat } n \text{ do } b) \subseteq \Psi(b)$.*

Proof. By induction on n, using the fact that repetition-unfolding steps emit no hazards. Details are given in Appendix A.9.

These lemmas extend the hazard-soundness guarantee of the core analysis to iteration.

Theorem 4.5 (Hazard Soundness for Iteration). *The extended type-and-effect system is hazard sound under prefix semantics.*

In particular, if $\Gamma \vdash$ *while* e *do* b : *unit* $\triangleright$ (H, r), *then* $\Psi(\text{while } e \text{ do } b) \subseteq H$.
Likewise, if $\Gamma \vdash$ *repeat* n *do* b : *unit* $\triangleright$ (H, r), *then* $\Psi(\text{repeat } n \text{ do } b) \subseteq H$.

Proof. By structural induction on the extended typing derivation. The core cases use Lemma 4.1, while the iteration cases use Lemmas 4.3 and 4.4. Details are given in Appendix A.10.

As in the core system, hazard soundness yields a quantitative guarantee independently of the loss model used in direct risk analysis.

Corollary 4.6 (Revaluation of Iteration Summaries). *Suppose an iterative expression* e_{it} *is assigned* $\Gamma \vdash e_{\text{it}}$: *unit* $\triangleright$ (H, r), *where* e_{it} *is either a* `while` *expression or a bounded* `repeat`.
Then, for every monotone loss model $L' : \mathcal{P}(W) \to X_\infty$, $R_{L'}(e_{\text{it}}) \preceq L'(H)$.

Proof. By Theorem 4.5 and Lemma 4.2.

For direct quantitative soundness, we use the prefix analogue of Lemma 2.1.

Lemma 4.7 (Hazards of Concatenated Prefixes). *For any composable finite execution segments* σ_1 *and* σ_2, $\mathsf{hazards}(\sigma_1 \cdot \sigma_2) = \mathsf{hazards}(\sigma_1) \cup \mathsf{hazards}(\sigma_2)$.

Proof. Immediate from the definition of $\mathsf{hazards}(\sigma)$. Details are given in Appendix A.11.

Theorem 4.8 (Direct Risk Soundness for Iteration). *Fix a loss model* L *satisfying Assumption 2.2, and suppose primitive annotations satisfy Assumption 3.1.*
The extended type-and-effect system is risk sound under prefix semantics.
In particular, if $\Gamma \vdash$ *while* e *do* b : *unit* $\triangleright$ (H, r_ℓ), *then* $R_L(\text{while } e \text{ do } b) \preceq r_\ell$.
Likewise, if $\Gamma \vdash$ *repeat* n *do* b : *unit* $\triangleright$ $(H, n \cdot r_b)$, *then* $R_L(\text{repeat } n \text{ do } b) \preceq n \cdot r_b$.

Proof. By structural induction on the extended typing derivation, using prefix soundness of immediate subexpressions.

The core cases follow from Lemma 4.1. For unbounded iteration, induction over finite loop unfoldings uses the closure premise of T-While. For bounded repetition, induction proceeds on n.

In both iteration cases, Lemma 4.7 and Assumption 2.2 bound the loss of composed execution segments. Details are given in Appendix A.12.

The hazard and quantitative components therefore retain their complementary roles in the presence of iteration.

The hazard component provides a reusable behavioral summary that can be revalued under alternative monotone consequence models. The directly inferred risk component provides a sound, potentially more precise bound for the loss model and primitive annotations used by the current analysis.

As in the core system, the two bounds are not necessarily ordered relative to one another, and failure to establish a bound within a policy threshold does not imply that a concrete execution violates the policy.

5 Extension: Assumption-Guarded Symbolic Risk Bounds

Section 4 assigns the bounded repetition repeat n do b the quantitative bound $n \cdot r_b$, where $n \in \mathbb{N}$. In many applications, however, the repetition count is not statically fixed. It may depend on an input size, batch size, deployment limit, or other environmental parameter.

We extend the quantitative layer of the analysis with symbolic parameters. The extension allows quantitative summaries to represent families of concrete bounds and supports policy certification under explicitly stated assumptions about those parameters.

For example, if a body has direct risk bound $(1, 1, 2)$ and n denotes an unknown repetition count, then repeat n do b is assigned the symbolic bound $(n, n, 2n)$. Rather than choosing a specific value

of n, we can check whether the symbolic bound satisfies a policy under an assumption such as $n \leq 10$.

The extension preserves the distinction between behavioral abstraction and quantitative valuation. Symbolic instantiation may change how often operations execute and therefore which hazards actually occur, but it does not change the hazard identities or behavioral annotations associated with primitive operations.

The analysis remains conservative across all admissible parameter valuations. In particular, the hazard component must safely approximate the behavior of every permitted concrete instance.

This extension is deliberately lightweight. It does not attempt to infer dependent refinements or synthesize environmental assumptions. Instead, it separates symbolic quantitative summaries from the external constraints under which those summaries satisfy a policy.

5.1 Symbolic Quantitative Expressions

Let $\mathcal{S}$ be a set of symbolic parameters. We extend quantitative components with expressions over $\mathcal{S}$.

The symbolic domain must be closed under the quantitative operations used by the typing rules, including constants, sequential composition, alternative join, and repetition.

For the concrete vector instantiation used throughout our examples, symbolic scalar expressions are generated by $a ::= c \mid s \mid a_1 + a_2 \mid \max(a_1, a_2) \mid a_1 a_2 \mid \infty$, where $c \in \mathbb{R}_{\geq 0}$ and $s \in \mathcal{S}$.

A symbolic risk vector is a d-tuple of such expressions. We write X_{sym} for the resulting symbolic quantitative domain.

The grammar describes the symbolic operations needed by the analysis rather than prescribing a particular decision procedure. An implementation may restrict expressions to a decidable fragment, such as affine arithmetic with maxima and restricted products.

Admissible valuations. A valuation assigns concrete values to parameters: $\nu : \mathcal{S} \to \mathbb{R}_{\geq 0}$.

We distinguish parameters used as repetition counts from general quantitative parameters. Let $\mathcal{S}_{\mathbb{N}} \subseteq \mathcal{S}$ denote the designated natural-valued parameters. A valuation is *admissible* only if $\nu(s) \in \mathbb{N}$ for every $s \in \mathcal{S}_{\mathbb{N}}$.

Thus, if n occurs as the repetition count in repeat n do b, then $\nu(n) \in \mathbb{N}$.

Valuation extends recursively to symbolic expressions and pointwise to symbolic risk vectors.

For a symbolic risk vector r, we write $\nu(r) \in X_\infty$ for its concrete quantitative interpretation.

For the additive-vector instantiation, valuation interprets + as addition, max as componentwise join, and multiplication as ordinary nonnegative multiplication.

As in Section 4, repetition is interpreted through iterated quantitative composition: $0 \cdot r = \mathbf{0}$ and $(n+1) \cdot r = r \oplus (n \cdot r)$.

For extended nonnegative vectors, we use $0 \cdot \infty = 0$ in this iterated-composition interpretation, ensuring that zero repetitions incur zero quantitative effect.

Consequently, valuation satisfies $\nu(n \cdot r) = \nu(n) \cdot \nu(r)$ for well-formed repetition expressions.

Symbolic ordering. The ordering of symbolic quantitative expressions is defined by their concrete interpretations.

For symbolic risk vectors r_1, r_2, write $r_1 \preceq_{\mathsf{sym}} r_2$ if, for every admissible valuation ν, $\nu(r_1) \preceq \nu(r_2)$.

Thus, symbolic ordering denotes a universally valid quantitative inequality rather than an ordering of expression syntax.

Valuation is compatible with the quantitative operations. In particular, $\nu(r_1 \oplus r_2) = \nu(r_1) \oplus \nu(r_2)$ and $\nu(r_1 \sqcup r_2) = \nu(r_1) \sqcup \nu(r_2)$.

Moreover, $r_1 \preceq_{\mathsf{sym}} r_2$ implies $\nu(r_1) \preceq \nu(r_2)$ for every admissible valuation ν.

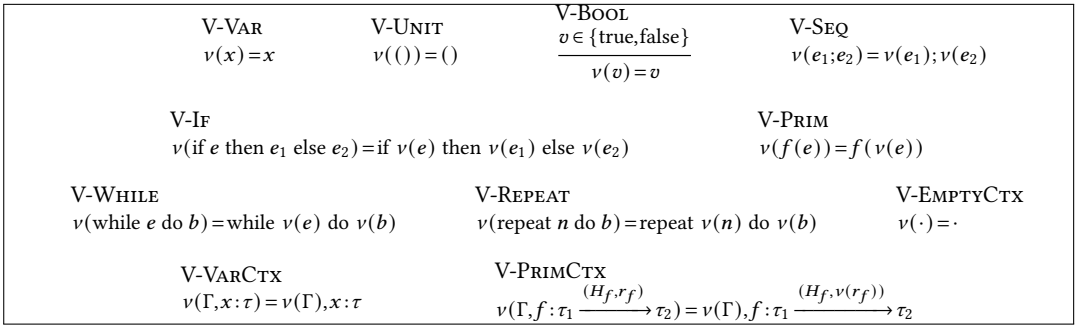


Fig. 4. Valuation of expressions and typing contexts. Valuation instantiates symbolic quantitative information and repetition counts while preserving primitive hazard annotations.

These properties ensure that symbolic composition followed by valuation agrees with valuation followed by concrete quantitative composition.

In particular, any symbolic inequality used as a typing-rule side condition, including the pre-fixed-point condition of T-While, is interpreted through $\preceq_{\text{sym}}$.

Hence such side conditions remain valid under every admissible concrete valuation. Environmental assumptions will be used separately for policy certification.

5.2 Valuation of Programs and Typing Contexts

Symbolic parameters may occur in repetition bounds and primitive quantitative annotations. We therefore extend valuation structurally to expressions and typing contexts, as shown in Figure 4.

Valuation does not substitute ordinary program variables. Its effect on program syntax is to instantiate symbolic repetition counts. In typing contexts, it instantiates symbolic quantitative annotations.

The primitive hazard annotations remain unchanged.

In particular, $v(H_f) = H_f$.

This reflects the separation established in Sections 2 and 3: symbolic instantiation changes quantitative parameters and possibly the number of primitive executions, but it does not change the hazard identities associated with those primitives.

Consequently, the same conservative hazard summary may remain valid across the family of concrete programs obtained through admissible valuation.

5.3 Assumption Contexts and Entailment

An assumption context Φ is a finite set of constraints over symbolic parameters.

These constraints describe externally supplied facts about the environment in which the program executes. Examples include input-size limits, deployment configurations, batch-size restrictions, and relationships among workflow parameters.

A valuation v satisfies Φ, written $v \models \Phi$, when v is admissible and every constraint in Φ evaluates to true.

The assumption context belongs to the certification layer rather than the operational semantics.

It does not determine how primitive hazards are generated. Instead, it restricts the set of concrete parameter valuations for which a symbolic risk bound is certified.

Assumption-guarded entailment. We write $\Phi \models r \preceq \rho$ when every admissible valuation satisfying Φ instantiates the symbolic risk bound below the policy threshold ρ.

Formally, $\Phi \models r \preceq \rho$ if and only if, for every admissible ν such that $\nu \models \Phi$, we have $\nu(r) \preceq \rho$.

Unlike the unconditional symbolic ordering $\preceq_{\mathsf{sym}}$, this relation permits inequalities that hold only under specific environmental assumptions.

For example, the symbolic inequality $n \leq 10$ need not hold under all admissible valuations. However, it follows from the assumption context $\Phi = \{n \leq 10\}$.

To avoid vacuous certification, we require an assumption context used for certification to be satisfiable: there must exist at least one admissible valuation ν such that $\nu \models \Phi$.

Satisfiability does not establish that a particular deployment actually satisfies Φ. The assumptions must be justified by external evidence or enforcement mechanisms for the resulting certificate to apply to that deployment.

Sound symbolic primitive annotations. Primitive quantitative annotations may themselves contain symbolic parameters. Their soundness must therefore hold throughout the valuation space permitted by Φ.

Definition 5.1 (Φ-Sound Primitive Annotations). *A symbolic typing context Γ is* Φ-sound with respect to L *if, for every admissible valuation $\nu \models \Phi$ and every primitive annotation $f : \tau_1 \xrightarrow{(H_f, r_f)} \tau_2 \in \Gamma$, we have $L(H_f) \preceq \nu(r_f)$.*

This is the symbolic analogue of Assumption 3.1.

It ensures that symbolic primitive annotations represent families of conservative quantitative bounds rather than unconstrained numerical expressions.

The requirement is relative to the selected loss model L. Changing the loss model may therefore require revalidation of primitive quantitative annotations even when their behavioral hazard summaries remain valid.

5.4 Assumption-Guarded Policy Certification

Typing judgments are lifted to symbolic quantitative bounds: $\Phi; \Gamma \vdash e : \tau \triangleright (H, r)$, where $r \in X_{\mathsf{sym}}$.

The assumption context is carried with the judgment, but the effect-propagation rules remain those of Sections 3 and 4.

Quantitative operations are interpreted symbolically, and side conditions inherited from the concrete typing rules must hold under the unconditional symbolic ordering $\preceq_{\mathsf{sym}}$.

In particular, the bounded-repetition rule is extended to symbolic repetition counts as follows.

$$\textsc{T-Repeat-Sym}\quad \frac{\Phi; \Gamma \vdash b : \mathrm{unit} \triangleright (H_b, r_b)}{\Phi; \Gamma \vdash \mathrm{repeat}\ n\ \mathrm{do}\ b : \mathrm{unit} \triangleright (H_b, n \cdot r_b)}$$

Here, n is a concrete natural number or a designated natural-valued symbolic parameter.

The risk component $n \cdot r_b$ denotes symbolic repeated composition. Its concrete interpretation under ν is $\nu(n) \cdot \nu(r_b)$, as defined in Section 4.

The assumptions in Φ do not otherwise affect effect propagation. They are consulted when a symbolic summary is compared against a quantitative policy.

To distinguish static certification from semantic policy satisfaction, we introduce the following certification rule.

$$\textsc{Cert-Assume}\quad \frac{\Phi; \Gamma \vdash e : \tau \triangleright (H, r) \quad \Phi \models r \preceq \rho \quad \exists \nu.\ \nu \models \Phi}{\Phi; \Gamma \vdash e : \tau\ \mathsf{cert}_\rho}$$

Workflow

```
setup();
repeat n do process();
if c then
  repeat m do validate()
else
  repeat n do validate()
```

Primitive risks

`setup`	$(1, 0, 2)$
`process`	$(0, 1, 2)$
`validate`	$(1, 0, 3)$

Fig. 5. Parameterized data-processing workflow and primitive quantitative summaries.

The judgment $\Phi; \Gamma \vdash e : \tau \ \mathrm{cert}_\rho$ states that the inferred symbolic bound satisfies ρ for every admissible parameter valuation consistent with the satisfiable assumption context Φ.

This generalizes concrete certification. If Φ fixes the values of all parameters, checking reduces to an ordinary quantitative comparison after substitution.

If Φ instead supplies inequalities or relationships among parameters, certification applies uniformly to the entire permitted family of instances.

The guarantee remains conditional on the assumed primitive annotations and on the environment satisfying Φ.

5.5 Example: Parameterized Data-Processing Workflow

Consider a workflow that processes a collection of records and optionally performs a more expensive validation stage.

As shown in Figure 5, the workflow first executes a setup step, then processes n records, and finally chooses between validating m selected records and validating all n records.

The parameters m, n are natural valued. We assume the conditional guard c is pure, so it contributes neither hazards nor quantitative risk.

The setup step contributes $(1, 0, 2)$.

The processing stage contributes $n \cdot (0, 1, 2) = (0, n, 2n)$.

The selected-record validation branch contributes $m \cdot (1, 0, 3) = (m, 0, 3m)$, while the full-validation branch contributes $n \cdot (1, 0, 3) = (n, 0, 3n)$.

The conditional therefore has bound $(m, 0, 3m) \sqcup (n, 0, 3n) = (\max(m, n), 0, \max(3m, 3n))$.

Combining the stages gives $r = (1, 0, 2) \oplus (0, n, 2n) \oplus (\max(m, n), 0, \max(3m, 3n))$.

Under componentwise addition, this simplifies to $r = (1 + \max(m, n), n, 2 + 2n + \max(3m, 3n))$.

Let the quantitative policy be $\rho = (11, 10, 32)$.

Without further assumptions on m and n, the symbolic bound cannot be certified against ρ.

Now consider the assumption context $\Phi = \{m \le n,\ n \le 10\}$.

Under $m \le n$, the inferred risk bound simplifies to $(1 + n, n, 2 + 5n)$.

The first two components remain within the policy for $n \le 10$, but the third component can reach 52. Consequently, $\Phi \not\models r \preceq \rho$.

For example, the admissible valuation $\nu(m) = \nu(n) = 10$ satisfies Φ but instantiates the bound to $\nu(r) = (11, 10, 52) \not\preceq (11, 10, 32)$.

Now strengthen the environmental assumption: $\Phi' = \{m \le n,\ n \le 6\}$.

Under Φ', the bound again simplifies to $(1+n, n, 2+5n)$. Because $n \le 6$, we have $(1+n, n, 2+5n) \preceq (7, 6, 32) \preceq (11, 10, 32)$.

Thus $\Phi' \models r \preceq \rho$, and the workflow is statically certified for every admissible valuation satisfying Φ'.

This example illustrates assumption-guarded certification: the type-and-effect analysis produces one compositional symbolic summary, while external constraints determine whether that summary is sufficient for a particular quantitative policy.

The checker does not establish the environmental assumptions themselves. Certification applies to a deployment only when those assumptions are satisfied.

5.6 Soundness of Assumption-Guarded Certification

We establish soundness by relating symbolic typing derivations to their concrete instances under admissible valuations.

The first lemma states that valuation preserves typing structure. Symbolic parameters in repetition bounds and primitive quantitative annotations are instantiated, while ordinary program variables, types, and hazard annotations remain unchanged.

Lemma 5.2 (Valuation Preservation). *If* $\Phi; \Gamma \vdash e : \tau \triangleright (H, r)$ *and* $\nu \models \Phi$, *then* $\nu(\Gamma) \vdash \nu(e) : \tau \triangleright (H, \nu(r))$.

Proof. By induction on the typing derivation. Valuation preserves the quantitative operations and all unconditional symbolic typing side conditions.

The bounded-repetition case additionally uses the requirement that repetition parameters instantiate to natural numbers. Details are given in Appendix A.13.

The next lemma makes explicit how symbolic primitive soundness supports the concrete soundness theorem.

Lemma 5.3 (Primitive Soundness under Valuation). *If* Γ *is* Φ*-sound with respect to* L *and* $\nu \models \Phi$, *then every primitive annotation in* $\nu(\Gamma)$ *satisfies Assumption 3.1.*

Proof. Immediate from Definition 5.1. Details are given in Appendix A.14.

We now define semantic policy satisfaction uniformly over the family of concrete instances permitted by Φ.

Definition 5.4 (Policy Satisfaction under Assumptions). *An expression* e *satisfies policy* ρ *under assumptions* Φ *and loss model* L, *written* $e \models_{L,\Phi} \rho$, *if, for every admissible valuation* $\nu \models \Phi$, $R_L(\nu(e)) \preceq \rho$.

This definition uses the prefix-based risk semantics of Section 4.

Theorem 5.5 (Soundness of Assumption-Guarded Policy Certification). *Suppose* $\Phi; \Gamma \vdash e : \tau \triangleright (H, r)$, Γ *is* Φ*-sound with respect to* L, *and* $\Phi \models r \preceq \rho$.

Then $e \models_{L,\Phi} \rho$. *Equivalently, for every admissible valuation* $\nu \models \Phi$, $R_L(\nu(e)) \preceq \rho$.

Proof. Let $\nu \models \Phi$ be arbitrary.

By Lemma 5.2, $\nu(\Gamma) \vdash \nu(e) : \tau \triangleright (H, \nu(r))$.

By Lemma 5.3, the instantiated primitive annotations satisfy the concrete primitive-soundness assumption.

The prefix-based risk-soundness theorem for the extended language therefore gives $R_L(\nu(e)) \preceq \nu(r)$.

Since $\Phi \models r \preceq \rho$ and $\nu \models \Phi$, we have $\nu(r) \preceq \rho$.

By transitivity, $R_L(\nu(e)) \preceq \rho$.

Because ν was arbitrary, the result holds for every admissible valuation satisfying Φ. Details are given in Appendix A.15.

The hazard component remains independent of the symbolic quantitative interpretation. Combining valuation preservation with prefix-based hazard soundness gives the following property.

Corollary 5.6 (Hazard Stability under Valuation). *If* $\Phi; \Gamma \vdash e : \tau \triangleright (H, r)$, *then, for every admissible* $\nu \models \Phi$, $\Psi(\nu(e)) \subseteq H$.

Proof. By Lemma 5.2, the instantiated expression $\nu(e)$ retains the hazard component H. Hazard soundness for the concrete extended language therefore gives $\Psi(\nu(e)) \subseteq H$.

Consequently, a symbolic typing derivation provides a uniform behavioral abstraction for the permitted concrete program instances.

By Lemma 4.2, this abstraction can be revalued under alternative monotone loss models. In particular, for any admissible $\nu \models \Phi$ and monotone $L' : \mathcal{P}(W) \to X_\infty$, we have $R_{L'}(\nu(e)) \preceq L'(H)$.

Thus, symbolic assumptions affect quantitative policy certification without eliminating the separation between behavioral abstraction and consequence valuation.

6 Extension: History-Sensitive Hazard Interpretation

The core system records hazards in a flat set. This abstraction is appropriate when a hazard can be associated directly with an execution step, but it cannot distinguish histories containing the same events in different orders or with different multiplicities. Consequently, it cannot directly recognize hazards such as repeated authentication failures or attaching a sensitive resource after public exposure has been enabled.

Ordered and history-sensitive effects are well established [20]. Our purpose is not to introduce ordered effects themselves. Instead, we introduce an abstract history domain as an intermediate layer between execution and quantitative consequence: $Q \xrightarrow{\eta} \mathcal{P}(W) \xrightarrow{L} X_\infty$.

The domain Q summarizes relevant execution histories, η recognizes hazards from those summaries, and L assigns quantitative consequences to the recognized hazards. Separating these interfaces permits the same behavioral history abstraction to support different hazard definitions and quantitative valuations.

The extension retains the primitive hazards of the core calculus while also allowing additional hazards to arise from combinations of events. We first define the history domain, then extend the typing rules and establish soundness and reinterpretation properties.

6.1 Abstract History Domains

An *abstract history domain* consists of $(Q, \preceq_Q, q_0, \otimes, \sqcup_Q)$, where elements of Q summarize finite execution histories.

The order $q \preceq_Q q'$ means that q' is at least as conservative a history summary as q. Depending on the application, the order may represent numeric ordering of counters, set inclusion over event flags, or inclusion over abstract protocol states.

The element $q_0 \in Q$ represents the empty history. We require q_0 to be the least element: $q_0 \preceq_Q q$ for every $q \in Q$.

Sequential composition $\otimes : Q \times Q \to Q$ combines histories of consecutively executed fragments. We assume $\otimes$ is associative, has q_0 as its identity, and is monotone in both arguments.

Thus, if $q_1 \preceq_Q q_1'$ and $q_2 \preceq_Q q_2'$, then $q_1 \otimes q_2 \preceq_Q q_1' \otimes q_2'$.

Because q_0 is least and is the identity of $\otimes$, these assumptions imply $q \preceq_Q q \otimes q'$ for all $q, q' \in Q$. This property is needed to soundly approximate execution prefixes that stop before the complete syntactic computation has finished.

The operator $\sqcup_Q : Q \times Q \to Q$ conservatively merges alternative execution histories. We require it to be monotone and to upper-bound both arguments: $q_1 \preceq_Q q_1 \sqcup_Q q_2$ and $q_2 \preceq_Q q_1 \sqcup_Q q_2$.

In common instantiations, $\sqcup_Q$ is the least upper bound under $\preceq_Q$.

Hazard interpretation. The abstract history domain describes behavior but does not itself determine which histories are hazardous. That interpretation is supplied separately by $\eta : Q \to \mathcal{P}(W)$.

We require η to be monotone: $q \preceq_Q q'$ implies $\eta(q) \subseteq \eta(q')$. We also assume $\eta(q_0) = \emptyset$.

$$\textsc{H-Var}\quad \Gamma, x:\tau \vdash x:\tau \triangleright (q_0,\mathbf{0}) \qquad \textsc{H-Unit}\quad \Gamma \vdash ():\mathrm{unit} \triangleright (q_0,\mathbf{0}) \qquad \textsc{H-Bool}\quad \frac{v \in \{\mathrm{true},\mathrm{false}\}}{\Gamma \vdash v:\mathrm{bool} \triangleright (q_0,\mathbf{0})}$$

$$\textsc{H-Seq}\quad \frac{\Gamma \vdash e_1:\mathrm{unit} \triangleright (q_1,r_1) \quad \Gamma \vdash e_2:\tau_2 \triangleright (q_2,r_2)}{\Gamma \vdash e_1;e_2:\tau_2 \triangleright (q_1 \otimes q_2,\ (r_1 \oplus r_2) \sqcup L(\eta(q_1 \otimes q_2)))}$$

$$\textsc{H-If}\quad \frac{\Gamma \vdash e:\mathrm{bool} \triangleright (q,r) \quad \Gamma \vdash e_1:\tau \triangleright (q_1,r_1) \quad \Gamma \vdash e_2:\tau \triangleright (q_2,r_2)}{\Gamma \vdash \mathrm{if}\ e\ \mathrm{then}\ e_1\ \mathrm{else}\ e_2:\tau \triangleright \big(q \otimes (q_1 \sqcup_Q q_2),\ \big(r \oplus (r_1 \sqcup r_2)\big) \sqcup L\big(\eta\big(q \otimes (q_1 \sqcup_Q q_2)\big)\big)\big)}$$

$$\textsc{H-Prim}\quad \frac{f:\tau_1 \xrightarrow{(H_f,q_f,r_f)} \tau_2 \in \Gamma \quad \Gamma \vdash e:\tau_1 \triangleright (q,r)}{\Gamma \vdash f(e):\tau_2 \triangleright \big(q \otimes q_f,\ (r \oplus r_f) \sqcup L(\eta(q \otimes q_f))\big)}$$

Fig. 6. History-aware typing rules. History summaries are composed before hazards are extracted through η and quantitatively interpreted through L.

The history-sensitive system extends the core semantics rather than discarding its primitive hazards. Accordingly, a primitive has an enriched annotation $f : \tau_1 \xrightarrow{(H_f,q_f,r_f)} \tau_2$, where H_f is its directly emitted hazard set, q_f is its history contribution, and r_f is its direct quantitative annotation.

The flat core annotation is recovered by projecting away q_f. Conversely, the history-aware typing rules use q_f and r_f while retaining H_f in the operational semantics.

To ensure that history interpretation preserves the directly emitted hazards, we require $H_f \subseteq \eta(q_f)$ for every primitive f.

This compatibility condition permits η to recognize additional compound hazards without forgetting hazards already present in the core execution semantics.

Different applications may use the same history domain and composition operators with different hazard interpretations. Similarly, the hazards recognized by η may be assigned different quantitative consequences by different monotone loss models $L : \mathcal{P}(W) \to X_\infty$.

As in the preceding sections, soundness of the resulting quantitative analysis is conditional on the adequacy of the chosen annotations and interpretations.

6.2 History-Aware Typing Judgments

We extend typing judgments to record abstract history summaries: $\Gamma \vdash e : \tau \triangleright (q, r)$, where $q \in Q$ conservatively summarizes possible execution histories and $r \in X_\infty$ is a quantitative risk bound.

Unlike the flat core judgment, the behavioral component now retains information about history. The corresponding hazard summary is obtained through $\eta(q)$.

Thus, q represents abstract execution behavior, η determines which hazards are recognized, and L assigns their quantitative consequences.

A compound hazard may become recognizable only after two history summaries are composed. For example, it is possible that $\eta(q_1) = \eta(q_2) = \emptyset$ while $\eta(q_1 \otimes q_2) \neq \emptyset$.

The typing rules must therefore account for $L(\eta(q))$ after history composition. We maintain the invariant $L(\eta(q)) \preceq r$, established in Lemma 6.5.

Figure 6 gives the resulting rules.

The original primitive hazard set H_f remains part of the operational semantics. In particular, executing a primitive under the enriched annotation still emits H_f, as in rule E-Prim of Figure 2.

The history contribution q_f need not itself induce an additional compound hazard. For example, a primitive may record that authentication has occurred, while a later operation produces a history that triggers a hazard recognized by η.

The quantitative annotation r_f retains its role as a direct conservative bound. For consistency with Section 3, we continue to require $L(H_f) \preceq r_f$.

Conservatism of branch joins. The conditional rule combines alternative histories using $q_1 \sqcup_Q q_2$ before applying η. Consequently, the joined history may recognize a hazard that is not realized by every concrete execution of the conditional.

More precise path-sensitive history abstractions may reduce this over-approximation. The soundness results require only that the joined summary upper-bound both alternatives.

The direct quantitative component also retains the locally supplied primitive bounds. However, unlike the flat core analysis, the history-aware rules explicitly include $L(\eta(q))$ in the inferred risk component. Therefore, the resulting r is not necessarily more precise than revaluing the history summary. Its benefit is that it combines the propagated primitive bounds with the consequences recognized from composed histories.

6.3 Semantic Interpretation of Histories

We relate the static history summaries to finite execution prefixes.

Let Step denote the evaluation steps generated by the operational semantics. Each step has an abstract history contribution, determined by a function $\kappa : \mathrm{Step} \to Q$.

When a step executes a primitive with annotation $f : \tau_1 \xrightarrow{(H_f, q_f, r_f)} \tau_2$, we define its contribution to be q_f. Pure evaluation steps contribute q_0.

This definition also applies to primitive reductions propagated through evaluation contexts. For example, if an evaluation-context step $E[f(v)] \to E[v']$ is induced by a primitive reduction $f(v) \to v'$, its history contribution is q_f, not q_0.

Thus, κ depends on the underlying reduction rule responsible for the step. Evaluation-context propagation does not discard the history contribution of a primitive execution.

Let $\sigma = e_0 \to e_1 \to \cdots \to e_n$ be a finite execution prefix, and let s_i denote the step $e_i \to e_{i+1}$.

Its abstract runtime history is $q_\sigma = \kappa(s_0) \otimes \kappa(s_1) \otimes \cdots \otimes \kappa(s_{n-1})$. For the empty prefix, $q_\sigma = q_0$.

The hazards recognized from the prefix are $\mathsf{hazards}_\eta(\sigma) = \eta(q_\sigma)$.

Making the dependence on both hazard interpretation and valuation explicit, define $R_{\eta,L}(\sigma) = L(\eta(q_\sigma))$ and $R_{\eta,L}(e) = \bigsqcup_{\sigma \in \mathsf{Prefixes}(e)} R_{\eta,L}(\sigma)$.

The quantitative domain is X_∞, as in Section 4, so the supremum is well defined.

Relationship to the core semantics. The history-sensitive semantics retains the primitive hazard labels of Section 3. Its hazard interpretation may additionally recognize compound hazards that do not arise from any individual primitive step.

Let $\mathsf{hazards}(\sigma)$ denote the union of the directly emitted primitive hazards along σ, as defined in Section 4.

Because each primitive satisfies $H_f \subseteq \eta(q_f)$, and because $q_f \preceq_Q q_\sigma$ whenever the corresponding primitive occurs in σ, monotonicity of η gives $\mathsf{hazards}(\sigma) \subseteq \eta(q_\sigma)$.

Consequently, for the same loss model L, monotonicity implies $L(\mathsf{hazards}(\sigma)) \preceq L(\eta(q_\sigma))$.

Taking the supremum over execution prefixes yields $R_L(e) \preceq R_{\eta,L}(e)$.

Thus, a sound upper bound on $R_{\eta,L}(e)$ also bounds the original flat-hazard risk. The additional hazards recognized by η may make the history-sensitive risk strictly larger.

This comparison concerns the semantic loss model defined over recognized hazards. If an application instead intends to add a separate accumulated execution-cost metric to the history-induced losses, that combined objective requires an explicit semantic definition and corresponding soundness argument.

Lemma 6.1 (Compositionality of Runtime History). *For any composable execution prefixes σ_1 and σ_2, $q_{\sigma_1 \cdot \sigma_2} = q_{\sigma_1} \otimes q_{\sigma_2}$.*

Proof. By associativity of $\otimes$ and the definition of runtime history as a fold over step contributions. Details are given in Appendix A.16.

6.4 Example: Counting-Based History Abstraction

Consider a hazard activated only after repeated occurrences of an event, such as multiple failed access attempts.

A flat hazard set cannot distinguish one occurrence from several occurrences. We therefore instantiate the history domain with a counter.

Let $Q = \mathbb{N}$, ordered by the usual numeric order, with $q_0 = 0$. Sequential composition adds event counts: $q_1 \otimes q_2 = q_1 + q_2$. Branch join takes their maximum: $q_1 \sqcup_Q q_2 = \max(q_1, q_2)$.

These operations satisfy the required history-domain assumptions.

Suppose the primitive `fail` contributes $q_{\texttt{fail}} = 1$, while unrelated operations contribute 0. For simplicity, assume `fail` has no directly emitted core hazard; the additional hazard is recognized only after repeated occurrences.

Let $w_{\text{multi}} \in W$ represent the hazard of observing at least two relevant failures. Define $\eta(q) = \{w_{\text{multi}}\}$ when $q \geq 2$, and $\eta(q) = \emptyset$ otherwise.

Consider $e_1 = \texttt{fail}(); \texttt{fail}()$.

Each invocation contributes history 1. Sequential composition therefore gives $1 \otimes 1 = 2$.

Although $\eta(1) = \emptyset$, we have $\eta(2) = \{w_{\text{multi}}\}$.

Consequently, the typing rule derives a quantitative bound containing $L(\{w_{\text{multi}}\})$, specifically $(r_{\texttt{fail}} \oplus r_{\texttt{fail}}) \sqcup L(\{w_{\text{multi}}\})$.

Neither invocation individually induces the repeated-failure hazard. The hazard is recognized only after their history summaries have been composed.

Branching. Consider $e_2 =$ `if c then fail(); fail() else fail()`.

The then-branch has history summary 2, while the else-branch has summary 1. Assume the guard contributes $q_0 = 0$.

The conditional therefore has summary $q = 0 + \max(2, 1) = 2$, so $\eta(q) = \{w_{\text{multi}}\}$.

The branch-specific quantitative bounds are $r_{\text{then}} = (r_{\texttt{fail}} \oplus r_{\texttt{fail}}) \sqcup L(\eta(2))$ and $r_{\text{else}} = r_{\texttt{fail}} \sqcup L(\eta(1))$.

The conditional rule then gives $r = (r_c \oplus (r_{\text{then}} \sqcup r_{\text{else}})) \sqcup L(\eta(q))$.

Since $q = 2$, the resulting bound includes $L(\{w_{\text{multi}}\})$.

This conservatism is intentional: the joined history records that the repeated-failure state is reachable through at least one branch, even though it is not reached through every execution.

Order-sensitive histories. The counter abstraction distinguishes event multiplicity but not event ordering. To illustrate that the general framework also supports ordered histories, consider a different instantiation.

Let Σ be a finite alphabet of relevant events. Define Q as the set of prefix-closed languages over Σ containing the empty word ϵ.

Order these languages by inclusion. Let $q_0 = \{\epsilon\}$, let $\otimes$ be language concatenation, and let $\sqcup_Q$ be language union.

The empty-history element q_0 is least and is the identity of concatenation. The operators are associative and monotone, and union upper-bounds both alternatives.

For events pub and db, assign the primitive history contributions $q_{\text{pub}} = \{\epsilon, \text{pub}\}$ and $q_{\text{db}} = \{\epsilon, \text{db}\}$.

Define an interpretation η_{ord} that recognizes a hazard w_{ord} whenever a word in the abstract language contains pub followed by db in that order.

Then $w_{\text{ord}} \in \eta_{\text{ord}}(q_{\text{pub}} \otimes q_{\text{db}})$, but $w_{\text{ord}} \notin \eta_{\text{ord}}(q_{\text{db}} \otimes q_{\text{pub}})$.

This example demonstrates that ordering can be retained when the chosen history domain distinguishes sequences. It does not require a change to the general typing rules.

The counter and language abstractions illustrate different precision choices. The selection of an appropriate Q remains application dependent.

6.5 Revisiting the Motivating Example

We return to the deployment workflow from Section 1.1.

The core analysis of Section 3.4 derives the direct quantitative bound $(4, 4, 10)$ for the approved branch. That calculation accounts for locally supplied primitive summaries but does not assign an additional consequence to the combination of public ingress and customer-database attachment.

To recognize this compound hazard while retaining the original primitive hazards, instantiate the history domain as $Q = \mathcal{P}(W) \times \mathcal{P}(\{\text{pub}, \text{db}\})$.

The first component records the directly emitted hazard identities. The second records the two behavioral facts relevant to the compound hazard.

Order Q componentwise by set inclusion, take $q_0 = (\emptyset, \emptyset)$, and define both $\otimes$ and $\sqcup_Q$ by componentwise set union.

These operations satisfy all the abstract history-domain requirements.

For each primitive f, let its history contribution be $q_f = (H_f, F_f)$, where H_f is its original core hazard annotation and F_f records relevant behavioral flags.

The primitive `create_public_ingress` contributes $F_f = \{\text{pub}\}$, while `attach_customer_db` contributes $F_f = \{\text{db}\}$. Other primitives contribute $F_f = \emptyset$.

For $q = (H, F)$, define $\eta(q) = H \cup C(F)$, where $C(F) = \{w_{\text{pubdb}}\}$ if $\{\text{pub}, \text{db}\} \subseteq F$, and $C(F) = \emptyset$ otherwise.

The function η is monotone, satisfies $\eta(q_0) = \emptyset$, and preserves all directly emitted hazards.

Thus, the two relevant operations contribute independent behavioral facts, and their composition triggers the compound hazard w_{pubdb}.

Suppose $L(\{w_{\text{pubdb}}\}) = (8, 0, 0)$.

To make the quantitative calculation concrete, consider a loss model that aggregates individual hazard severities by componentwise maximum: $L(H) = \bigsqcup_{w \in H} L(\{w\})$, with $L(\emptyset) = \mathbf{0}$.

Such a model is monotone and is compatible with additive quantitative composition, as required in Section 2.

Assume the individual losses of the original primitive hazards are bounded by their corresponding primitive quantitative annotations, so their combined maximum is conservatively bounded by the previously inferred direct risk $(4, 4, 10)$.

The additional compound hazard contributes $(8, 0, 0)$. The history-aware analysis therefore obtains the conservative bound $(4, 4, 10) \sqcup (8, 0, 0) = (8, 4, 10)$.

Since $(8, 4, 10) \not\leq (5, 5, 12)$, the workflow cannot be certified against the deployment policy.

In particular, the first quantitative component exceeds its threshold.

The example illustrates the separation between behavioral facts, recognized hazards, and quantitative consequences. The two operations first contribute history information. Their combination is recognized through η, and the compound hazard is valued by L.

The example uses a co-occurrence condition rather than an ordering condition. An order-sensitive history domain, such as the one illustrated above, could distinguish whether public exposure preceded database attachment.

Neither the program nor the inferred history summary must change if the organization revises the quantitative consequence assigned to the compound hazard.

More generally, the same history summary may be interpreted under a different monotone hazard extraction function, provided the underlying behavioral abstraction remains appropriate. We formalize this property next.

6.6 Soundness and Reinterpretation

We relate statically inferred histories to the abstract histories induced by concrete execution prefixes.

Lemma 6.2 (History Soundness). *If* $\Gamma \vdash e : \tau \triangleright (q, r)$, *then* $q_\sigma \preceq_Q q$ *for every* $\sigma \in \mathsf{Prefixes}(e)$.

Proof. By induction on the typing derivation. Sequential composition uses Lemma 6.1 and monotonicity of $\otimes$.

Prefixes stopping within an earlier subexpression are covered by $q_0 \preceq_Q q$ and the monotonicity of sequential composition. The conditional case additionally uses the upper-bound property of $\sqcup_Q$.

Details are given in Appendix A.17.

Because η is monotone, history soundness immediately implies soundness of the recognized hazards.

Theorem 6.3 (Hazard Soundness with History). *If* $\Gamma \vdash e : \tau \triangleright (q, r)$, *then* $\bigcup_{\sigma \in \mathsf{Prefixes}(e)} \eta(q_\sigma) \subseteq \eta(q)$.

Proof. For every prefix σ, Lemma 6.2 gives $q_\sigma \preceq_Q q$. Monotonicity of η yields $\eta(q_\sigma) \subseteq \eta(q)$. Taking the union over all prefixes proves the result.

Details are given in Appendix A.18.

Reinterpretation. The history summary supports a richer form of reinterpretation than the flat hazard summary of Section 2.

Once a history summary q has been computed soundly, both the hazard interpretation η and the loss model L can be changed without recomputing the behavioral history abstraction.

Theorem 6.4 (History-Parametric Reinterpretation). *Suppose* $\Gamma \vdash e : \tau \triangleright (q, r)$.
Let $\eta' : Q \to \mathcal{P}(W)$ *be any monotone hazard interpretation satisfying* $\eta'(q_0) = \emptyset$, *and let* $L' : \mathcal{P}(W) \to X_\infty$ *be any monotone loss model.*
Define $R_{\eta', L'}(e) = \bigsqcup_{\sigma \in \mathsf{Prefixes}(e)} L'(\eta'(q_\sigma))$.
Then $R_{\eta', L'}(e) \preceq L'(\eta'(q))$.

Proof. For every execution prefix σ, Lemma 6.2 gives $q_\sigma \preceq_Q q$.

Monotonicity of η' gives $\eta'(q_\sigma) \subseteq \eta'(q)$. Monotonicity of L' therefore yields $L'(\eta'(q_\sigma)) \preceq L'(\eta'(q))$.

Taking the supremum over all finite prefixes establishes the result.

Details are given in Appendix A.19.

The theorem is parametric in the hazard interpretation and loss model. It does not require the original quantitative component r to remain sound under the alternative interpretation.

The guarantee concerns the newly interpreted semantic risk $R_{\eta', L'}(e)$.

If the alternative interpretation is also intended to cover every directly emitted core hazard, it must preserve the corresponding compatibility condition: $H_f \subseteq \eta'(q_f)$ for every primitive.

Under this additional condition, the same argument establishing $R_L(e) \preceq R_{\eta, L}(e)$ also applies to the alternative loss interpretation.

Thus, the history summary can be reused for changed hazard definitions or consequence models, while the soundness of a particular certification remains relative to the chosen interpretation.

Direct quantitative soundness. The directly inferred risk component depends on the particular η and L used during typing.

The following consistency property relates this component to the interpreted history summary.

Lemma 6.5 (History-Judgment Consistency). *If* $\Gamma \vdash e : \tau \triangleright (q, r)$, *then* $L(\eta(q)) \preceq r$.

Proof. By case analysis on the final typing rule.

The value rules have summary $(q_0, \mathbf{0})$, and $L(\eta(q_0)) = \mathbf{0}$.

Each compositional rule explicitly joins its propagated quantitative bound with the loss extracted from its resulting history summary.

Hence $L(\eta(q)) \preceq r$. Details are given in Appendix A.20.

Theorem 6.6 (Direct Risk Soundness with History). *If* $\Gamma \vdash e : \tau \triangleright (q, r)$, *then* $R_{\eta,L}(e) \preceq r$.

Proof. For every execution prefix σ, Lemma 6.2 and monotonicity of η give $\eta(q_\sigma) \subseteq \eta(q)$.

Monotonicity of L therefore yields $L(\eta(q_\sigma)) \preceq L(\eta(q))$.

By Lemma 6.5, $L(\eta(q)) \preceq r$. Thus every prefix risk is bounded by r.

Taking the supremum over all finite prefixes gives $R_{\eta,L}(e) \preceq r$.

Details are given in Appendix A.21.

Since the history interpretation preserves the directly emitted primitive hazards, the result also implies $R_L(e) \preceq r$ for the corresponding flat-hazard semantics under the same loss model.

The history-sensitive system therefore provides two complementary forms of quantitative reasoning.

The inferred component r is a sound quantitative bound for the hazard and loss interpretations currently in force.

The history summary q is a reusable abstraction of execution behavior that can subsequently be interpreted under alternative hazard definitions and consequence models.

These guarantees rely on the soundness of the primitive annotations, the compatibility of the chosen history interpretation with the underlying operational events, and the monotonicity assumptions specified above.

7 Prototype Implementation and Case Study

To demonstrate the practical realization of the proposed framework, we implemented a static checker for a restricted fragment of the Workflow Description Language (WDL), a workflow language for task-based data processing [38]. The prototype separates structural analysis from quantitative interpretation. A workflow is analyzed to obtain a reusable behavioral summary, which can subsequently be evaluated under different loss models and quantitative policies without repeating the structural analysis.

Beyond the smaller examples used to exercise the core typing rules, we evaluate the framework on a constructed checkpointed batch-publication workflow. The study investigates three questions: whether the same behavioral summary supports different quantitative decisions, whether history-sensitive abstractions improve precision over simpler hazard summaries, and how certification depends on symbolic batch parameters and externally supplied quantitative assumptions.

The study is an offline analysis experiment, not a reproduced incident or an actual cloud deployment. Workflow tasks are synthetic stubs, and their behavioral annotations and quantitative consequences are supplied assumptions. The evaluation therefore tests the compositional analysis and reinterpretation mechanisms rather than the empirical accuracy of a particular risk model.

7.1 Architecture and Analysis Pipeline

The frontend uses the `miniwdl` 1.15.0 parser and typechecker for WDL 1.1, followed by a restricted translation to the core intermediate representation (IR). The translation preserves task dependencies, named arguments, output references, and array cardinalities. In particular, it does not treat textual statement order as execution order: WDL dependencies determine the permitted ordering of tasks. The supported fragment includes task calls, aliases, workflow outputs, array `scatter` constructs, and simple conditionals. A skipped WDL conditional is represented by the unit-valued alternative of the core calculus.

The checker separates structural analysis and quantitative evaluation. Structural analysis consumes the IR and a JSON sidecar specifying primitive hazard labels, abstract history events, resource identities, parameter bindings, and environmental assumptions. It produces a serializable effect tree containing type information, primitive contracts, abstract histories, recognized hazards, repetition counts, and provenance information. The tree preserves compositional structure rather than recording only a union of hazards.

Importantly, this phase does not consult numerical loss values or primitive quantitative bounds. Consequently, the resulting summary can be cached and reused under alternative quantitative interpretations.

A separate evaluation phase supplies a numerical profile containing primitive risk bounds r_f and a loss model L. The implementation supports nonnegative vector-valued losses, with componentwise addition for sequential composition and componentwise maximum for alternative control-flow paths. The loss models used in the study sum the losses of distinct hazard labels. They are monotone and satisfy the sequential compatibility condition of Assumption **??**.

Before evaluating a profile, the checker verifies that the model provides values for all required hazards and that primitive annotations satisfy $L(H_f) \preceq r_f$. History annotations also preserve directly emitted hazards through $H_f \subseteq \eta(q_f)$.

The evaluator follows the recursive history-aware typing rules of Figure 6. In particular, sequential composition computes $r_{\text{seq}} = (r_1 \oplus r_2) \sqcup L(\eta(q_1 \otimes q_2))$. The quantitative bounds of subexpressions may already incorporate history-induced losses, so the final result is not generally equivalent to applying a single history-loss join to the base risk at the root. History-induced loss is not treated as an additional sequential cost.

The resulting architecture supports two forms of reuse. Changing the quantitative loss model requires only reevaluation of the cached behavioral summary, while changing the policy threshold requires only a new comparison against the inferred bound. Neither operation requires reparsing the workflow or rediscovering its hazards. Changing the hazard recognizer η, however, currently requires a new structural analysis. Thus, the implementation demonstrates loss-parametric revaluation, while the more general history-interpretation reuse established in Section 6 remains a theoretical capability of the framework.

An earlier WDL-like frontend is retained for compatibility with regression examples, including the motivating deployment example. It is distinct from the standard-WDL frontend used in the case study.

7.2 Checkpointed Batch-Publication Workflow

The case study models a batch-processing workflow that stages input files, constructs a report, and publishes the resulting artifact. The workflow contains ten task definitions: manifest checking, staging-resource selection, batch upload, checkpoint sealing, shard validation, shard processing, report assembly, report revision, publication, and staging cleanup.

Only shard validation and processing are repeated using WDL `scatter` constructs. For a batch containing n files, the workflow therefore contains $8 + 2n$ task invocations.

The workflow uses a selected staging resource B and an assembled logical report R. A staging checkpoint records that the staged inputs have been declared complete. Publishing makes the current report available to downstream consumers. These operations introduce two potentially undesirable execution patterns.

History-sensitive hazards. We define the following hazards independently of their numerical consequences:

- h_{write}: a batch upload occurs after the staging checkpoint has been sealed, potentially invalidating the checkpoint.
- h_{revise}: a report is revised after publication, potentially requiring reconciliation of previously released results.

These labels identify undesirable execution patterns, not confirmed data corruption or downstream harm. Ten additional direct hazard labels represent task-local operational-recovery loss classes, one for each task kind. Repeated occurrences of the same task reuse its direct hazard label.

We construct four workflow variants using identical tasks, resource identities, and invocation counts. The *in-order* variant uploads before sealing and revises before publishing. The *write-after-seal* variant reverses the staging dependency; the *revise-after-publication* variant reverses the report dependency; and the *both-misordered* variant reverses both.

These variants differ in their dependency graphs, rather than merely in the textual order of WDL statements. For example, the in-order workflow makes the checkpoint-sealing task depend on the upload receipt. The corresponding misordered variant instead makes the upload depend on the sealing operation.

The analysis verifies that the operations refer to the same selected staging resource and logical report, including references propagated through supported aliases. These checks establish reference provenance but do not independently validate the behavior of opaque task implementations.

The supported fragment requires stateful operations to be dependency-ordered relative to other calls. Independent event-neutral worker operations may remain unordered. Stateful operations inside conditional or scattered regions, arbitrary concurrent mutation, retries, and interference from external writers are outside the supported model.

A fifth fixture deliberately omits a required dependency. The checker reports this workflow as *unsupported*, rather than rejecting it on quantitative policy grounds. This distinction prevents unsupported execution orders from being treated as successfully analyzed programs.

Concrete study inputs satisfy $1 \le n \le 20$, with n determined from the input-array cardinality. Input files must exist and be at most 1 MiB. Symbolic runs use assumptions defining an admissible batch-size interval. The input envelope and task contracts are conditions of certification, not runtime guarantees about file integrity or external services.

History-domain instantiation. The two compound hazards require the analysis to distinguish execution order. Let $E = \{s, u, p, v\}$ denote checkpoint sealing, upload, publication, and revision, respectively.

We instantiate the abstract history domain of Section 6 with summaries $q = (H, A)$, where H is a set of directly emitted hazards and A is a prefix-closed language of event sequences containing the empty word ϵ.

The history order is componentwise inclusion, with $q_0 = (\varnothing, \{\epsilon\})$. Sequential composition and branch join are defined by $(H, A) \otimes (K, C) = (H \cup K, AC)$ and $(H, A) \sqcup_Q (K, C) = (H \cup K, A \cup C)$, respectively.

A primitive contributing event e has the history component $(H_f, \{\epsilon, e\})$, while an event-neutral primitive contributes $(H_f, \{\epsilon\})$. The hazard interpretation η retains the direct hazards in H and additionally recognizes h_{write} when an event word contains s followed by u, and h_{revise} when it contains p followed by v.

The resulting domain satisfies the required history-algebra properties. Language concatenation is associative, monotone, and has $\{\epsilon\}$ as its identity; union conservatively combines alternative histories. Since every represented language contains ϵ, the empty history is least. Positive-pattern recognition is monotone, and direct hazards are preserved.

Importantly, the domain distinguishes $s;u$ from $u;s$ and $p;v$ from $v;p$. It therefore recognizes ordering-sensitive hazards without treating the mere co-occurrence of the corresponding operations as sufficient.

The implementation uses finite event languages for the admitted acyclic event structure. Repetition of event-neutral worker operations is supported, but repetition or unbounded looping of nonempty event histories is not. The domain recognizes positive patterns only: an additional operation cannot erase a hazard that has already occurred. This is consistent with the monotonicity assumptions of the formal model, although it does not capture recovery operations or hazards whose validity depends on the absence of subsequent events.

7.3 Loss Models and Quantitative Certification

To examine the separation between behavioral abstraction and quantitative interpretation, we evaluate the same workflows under two loss profiles and two policy thresholds.

The quantitative domain has three components: integrity-response analyst-hours, publication-rework operator-hours, and task-local operational-recovery operator-hours. All values are hypothetical allowances used for the experiment. They are not measured incident costs, elapsed execution times, billing charges, or expected losses.

The two profiles share all primitive behavior, event annotations, and hazard definitions. They differ in the consequences assigned to the two history-sensitive hazards:

Loss profile	$L(\{h_{\text{write}}\})$	$L(\{h_{\text{revise}}\})$
Standard	$(8, 0, 0)$	$(0, 4, 0)$
High assurance	$(20, 0, 0)$	$(0, 12, 0)$

Both profiles assign direct loss and primitive risk bound $(0, 0, 0.5)$ to each of the eight fixed task kinds, $(0, 0, 0.25)$ to shard validation, and $(0, 0, 1.75)$ to shard processing.

Under componentwise addition, the direct quantitative analysis derives the symbolic bound $r_{\text{base}}(n) = (0, 0, 4 + 2n)$.

The two policy thresholds are the routine policy $\rho_{\text{R}} = (10, 6, 24)$ and the contingency policy $\rho_{\text{C}} = (24, 16, 40)$.

Table 1 reports the history-sensitive bounds for a batch of five files. The reported results follow the recursive composition rules and incorporate the consequences of any recognized compound hazards.

Reinterpretation without reanalysis. The results directly illustrate the distinction between behavioral summaries, quantitative consequences, and policy requirements.

Consider the both-misordered workflow. Its abstract history and recognized hazard set are identical under both loss profiles. Under the standard profile, however, the inferred bound is $(8, 4, 14)$, whereas the high-assurance profile assigns $(20, 12, 14)$.

The standard profile certifies the workflow under the routine policy, while the high-assurance profile does not. The underlying workflow and its behavioral abstraction remain unchanged; only the interpretation of its hazards differs.

Table 1. History-sensitive risk bounds and policy decisions for $n = 5$. W denotes write after sealing, V denotes revision after publication, and WV denotes both misorderings. A indicates successful certification under the stated assumptions; N indicates failure to certify.

Variant	Loss profile	Inferred bound	Routine	Contingency
In order	Standard	$(0, 0, 14)$	A	A
In order	High assurance	$(0, 0, 14)$	A	A
W	Standard	$(8, 0, 14)$	A	A
W	High assurance	$(20, 0, 14)$	N	A
V	Standard	$(0, 4, 14)$	A	A
V	High assurance	$(0, 12, 14)$	N	A
WV	Standard	$(8, 4, 14)$	A	A
WV	High assurance	$(20, 12, 14)$	N	A

Alternatively, holding the high-assurance profile fixed and changing from the routine to the contingency policy changes the certification outcome without changing either the behavioral summary or the inferred quantitative bound.

Thus, the same analyzed behavior supports different quantitative decisions through independent changes to the loss and policy layers.

Successful certification of a workflow containing a recognized hazard does not imply that the hazard is absent. It means that the inferred consequence is within the selected policy threshold under the supplied assumptions.

Semantic loss and inferred bounds. The quantitative semantics of the paper assigns loss to sets of recognized hazards. Since hazard sets retain distinct labels rather than repeated occurrences, the semantic loss associated with all ten direct task labels is $(0, 0, 6)$, independently of the number of worker invocations.

For $n = 5$, the direct quantitative bound has operational component 14, which conservatively bounds this semantic loss. This difference reflects the precision of the chosen compositional abstraction: the inferred bound accumulates repeated primitive risk annotations, while the semantic loss function does not count duplicate hazard labels.

Accordingly, the quantity $4+2n$ is a conservative static bound for the stated hazard-loss objective, not a separate accumulated execution-cost metric.

7.4 Comparative Evaluation

We evaluate four workflow variants under four analysis modes, two loss profiles, two policies, and eleven batch configurations. The resulting matrix contains $4 \cdot 4 \cdot 2 \cdot 2 \cdot 11 = 704$ certification checks.

Seven configurations use concrete batch sizes $n \in \{1, 5, 10, 11, 18, 19, 20\}$. Four others use guarded symbolic intervals $1 \le n \le k$, where $k \in \{10, 11, 18, 19\}$.

Each workflow variant is parsed and lowered once. The principal matrix performs twelve structural analyses across the flat, flag-set, and ordered-history abstractions. The base-only mode reuses an ordered summary with history valuation disabled. All numerical comparisons subsequently reuse the corresponding structural summaries.

Comparison of behavioral abstractions. We compare four analysis modes.

The *base-only* mode propagates primitive quantitative bounds but ignores compound hazards. It therefore analyzes a reduced objective and cannot certify the complete hazard catalog.

Table 2. Comparison of analysis modes under the high-assurance profile and routine policy ($n = 5$). Base-only analyzes a reduced objective and does not certify the complete hazard catalog.

Analysis mode	In order	W	V	WV
Base-only	A	A	A	A
Conservative flat	N	N	N	N
Flag-set history	N	N	N	N
Ordered history	A	N	N	N

Table 3. Assumption-guarded certification for the in-order workflow with inferred bound $(0, 0, 4 + 2n)$.

Parameter assumption	Maximum bound	Routine	Contingency
$1 \le n \le 10$	24	A	A
$1 \le n \le 11$	26	N	A
$1 \le n \le 18$	40	N	A
$1 \le n \le 19$	42	N	N

The *conservative flat* mode assigns the post-seal-write hazard to each upload and the post-publication-revision hazard to each revision, regardless of execution order. The corresponding primitive bounds are strengthened to preserve the primitive-soundness condition.

The *flag-set* mode recognizes co-occurrence of sealing and upload, or publication and revision, without retaining their order.

Finally, the *ordered-history* mode uses the event-language abstraction described above to distinguish the relevant execution sequences.

Table 2 compares these approaches under the high-assurance loss profile, routine policy, and batch size $n = 5$.

The conservative flat and flag-set analyses infer $(20, 12, 14)$ for all four variants, while base-only infers $(0, 0, 14)$.

The ordered-history analysis instead certifies the in-order workflow while detecting the two undesirable ordering patterns. It therefore avoids unnecessary certification failures introduced by the coarser abstractions.

This demonstrates a precision advantage for the chosen history domain. It does not imply that all flat-hazard analyses necessarily miss these hazards; a conservative flat abstraction can recognize their possibility, but at the cost of rejecting executions that avoid the relevant ordering patterns.

Assumption-guarded certification. The same structural summary supports symbolic bounds over different admissible batch sizes.

For the in-order workflow, the inferred quantitative bound is $(0, 0, 4 + 2n)$. Under the routine policy, certification succeeds for all $1 \le n \le 10$, whereas the contingency policy permits all $1 \le n \le 18$.

Table 3 shows the corresponding boundary cases.

The symbolic checker establishes that the assumptions are satisfiable and that all admissible valuations satisfy the required quantitative inequality. The implementation uses conservative arithmetic entailment rather than a general-purpose symbolic solver. Contradictory or unsupported assumptions are not treated as valid certificates.

Importantly, failure to certify a symbolic bound does not establish a concrete semantic policy violation. For example, the operational component 26 at $n = 11$ exceeds the routine threshold of 24, but the semantic loss associated with the distinct direct hazard labels remains bounded by 6. The

failed check concerns the inferred quantitative bound, not a witnessed execution with unacceptable semantic loss.

Sensitivity analysis. To examine the dependence of certification outcomes on externally supplied quantitative assumptions, we perturb the two compound-hazard losses, the three policy components, and the ten primitive bounds.

Individual parameters are varied by −25% and +25% across the workflow variants, loss profiles, policies, and selected batch sizes. Primitive direct losses are scaled consistently with their corresponding bounds.

The experiment performs 1,440 individual-parameter checks and 96 combined favorable/adverse perturbations, for a total of 1,536 sensitivity checks. Certification succeeds in 861 individual cases and 51 combined cases.

The results illustrate how the quantitative decisions depend on supplied assumptions without requiring new structural analyses. For example, the write-after-seal loss of 20 under the high-assurance profile reaches the contingency security threshold of 24 at a multiplier of 1.2. A 25% increase therefore prevents certification.

These are controlled sensitivity scenarios, not empirical estimates of failure probabilities or the reliability of the workflow.

7.5 Validation, Reproducibility, and Limitations

The study includes an independent concrete monitor that enumerates dependency-respecting task schedules and task-boundary execution prefixes for $n \in \{1, 2, 3\}$.

The monitor shares the typed WDL dependency graph produced by `miniwdl`, but does not reuse the implementation of the abstract history domain or the quantitative evaluator.

Task attempts may succeed, fail before producing their abstract event, or fail after producing it. Dependent tasks are blocked by failed predecessors, while independent event-neutral worker tasks may continue.

The monitor records the hazards recognized along concrete prefixes and evaluates their losses through the same externally supplied quantitative profiles. It therefore provides a bounded consistency check between concrete execution behavior and the abstract analysis.

Across the four workflow variants, the monitor enumerates 388 successful schedules and 848 deduplicated reachable prefix states. Every observed hazard is included in the corresponding static history summary, and the concrete prefix losses remain bounded by both the revalued static history summary and the directly inferred quantitative bound.

No under-approximation was observed in these bounded experiments. Additional checks exercise short event sequences, sampled history-domain laws, and mutations of the event annotations. In particular, removing the abstract upload event causes the validation procedure to identify the resulting missing hazard.

The artifact includes workflow variants, synthetic inputs, annotation sidecars, numerical profiles, generated IR, dependency graphs, serialized effect summaries, and complete experimental results. The accompanying automated test suite contains 128 passing tests covering parsing, structural analysis, numerical evaluation, symbolic checking, diagnostics, and the case-study examples.

The experiments can be reproduced using the following commands:

```
uv sync --locked
uv run --locked python -m examples.run_case_study
uv run --locked python -m unittest discover -s tests
```

The dependencies are pinned, and the numerical experiments produce deterministic outputs. The artifact also records phase-specific timing samples for parsing, structural analysis, and numerical evaluation. These measurements describe the implementation's behavior on the constructed workflows; they do not establish scalability or end-to-end performance improvements for large workflow repositories.

Scope of the evidence. The case study provides evidence that the proposed semantic separation is useful for a class of workflow risks involving ordered task dependencies. The same abstract behavior can be valued under different loss models, checked against different policies, and reused across symbolic batch sizes. The comparison also demonstrates that retaining execution order can improve certification precision relative to conservative order-insensitive abstractions.

Nevertheless, the results remain conditional on the supplied task contracts, hazard definitions, resource identities, input restrictions, and quantitative assumptions. The experiment does not establish that the chosen loss values accurately represent real-world incidents.

The abstract-history operations satisfy the algebraic premises of the formal framework, but neither the Python implementation nor the WDL-to-core translation has been mechanically verified. The bounded concrete monitor provides additional validation, not a new general soundness theorem.

The prototype does not support arbitrary mutable state, stateful concurrency, external interference, retries, or unbounded repetition of nonempty event histories. Full WDL coverage, probabilistic consequences, and runtime-monitor integration remain outside the present implementation.

Within these limitations, the experiment demonstrates the practical value of maintaining distinct interfaces for behavioral abstraction, hazard recognition, quantitative loss, and policy certification.

8 Limitations and Future Directions

The framework provides sound compositional reasoning about hazards and quantitative consequences, but its guarantees are conditional on several modeling assumptions. We highlight the principal limitations and directions for extending the analysis.

Hazard and Loss Models. Primitive annotations, hazard interpretations, loss models, and policy thresholds are supplied externally. Our soundness results establish guarantees relative to these inputs; they do not establish that the inputs accurately represent a particular deployment environment. Developing reliable models may require domain expertise, security reviews, measurements, incident data, or other empirical evidence. An important practical direction is to incorporate validation and provenance of these annotations into the certification process.

Precision and History Abstraction. The analysis favors simple compositional summaries over path-sensitive reasoning. Consequently, joins may combine hazards from mutually exclusive executions, producing conservative bounds. Richer abstract domains, relational analysis, and path-sensitive techniques could improve precision. Similarly, the chosen history domain Q determines which temporal relationships can be represented. Counters and event flags capture repetition and co-occurrence but cannot express arbitrary temporal properties. The monotone history interpretation also records hazards cumulatively; representing conditions that cease to hold after recovery or state changes would require a richer semantic treatment. A systematic abstract-interpretation formulation could help derive and compare suitable history domains [8, 21].

Mutable State and Iteration. The core calculus is deliberately small and does not explicitly model assignments, mutable memory, or state-dependent control flow. Although primitives may return nondeterministic values, their operational semantics does not explicitly represent evolving program state. Consequently, state-dependent hazards and loop termination conditions cannot be analyzed precisely.

Extending the framework to mutable state would require a store-sensitive operational semantics and potentially richer abstractions for state-dependent hazards. The current treatment of unbounded iteration also uses conservative pre-fixed-point bounds, which may be infinite even when semantic loss is finite. State invariants and abstract interpretation could support more precise loop bounds. Furthermore, while the prototype implements iteration operations for specific history domains, the general history-aware calculus does not establish soundness for arbitrary history domains under unbounded iteration.

Symbolic and Value-Dependent Policies. The symbolic extension propagates quantitative expressions under externally supplied assumptions. It does not infer those assumptions or establish that they hold in a particular deployment. Nor does it provide the value-dependent inference capabilities of systems such as Liquid Resource Types [30]. Refinement types and constraint-based inference could strengthen the analysis by relating hazard conditions, program values, and quantitative bounds while retaining the separation between behavioral abstraction and consequence valuation.

Quantitative Scope. The current semantics considers conservative worst-case consequences over execution prefixes. It does not model probabilities, expected losses, or distributions of outcomes. In particular, rare high-consequence events receive the same treatment as frequently occurring ones. Probabilistic risk measures would require richer execution semantics and quantitative composition rules. Likewise, relational security properties, including quantitative information flow, are not directly captured by the present hazard-set semantics.

Implementation and Broader Applicability. The prototype supports a subset of WDL and demonstrates the integration of primitive annotations, history-sensitive hazards, symbolic bounds, and quantitative policy checking. It is not a complete WDL implementation, and the present evaluation does not establish scalability or comparative precision on large workflow repositories.

Extending the frontend and evaluating larger applications would provide stronger evidence of practical applicability. Concurrency, distributed execution, and adversarial interaction would also require richer behavioral models, particularly when hazards depend on interleavings or strategic choices. Finally, combining static certification with runtime monitoring could help validate environmental assumptions and detect behaviors outside the static model. These extensions remain future work.

9 Related Work

Our work builds on type-and-effect systems, quantitative program analysis, and formal reasoning about execution histories. Particularly relevant are resource-aware type systems, sequential and history-sensitive effect systems, and metric-aware service orchestration. We distinguish our contribution from these approaches by focusing on the explicit separation of behavioral abstraction, hazard recognition, quantitative valuation, and policy certification. We do not claim that quantitative effects, history-sensitive effects, or symbolic bounds are themselves novel, nor that the proposed framework subsumes existing quantitative analyses.

Type-and-effect systems and abstract interpretation. Type-and-effect systems extend ordinary typing judgments with compositional summaries of program behavior. Foundational work established polymorphic and generic effect systems [33, 35], while more recent developments demonstrate their applicability to a wide range of language properties and effect semantics [11, 19, 25, 44].

Our system adopts this established methodology rather than proposing a new general theory of effects. Its typing judgments expose two complementary forms of information: a behavioral summary of potentially hazardous execution and a quantitative bound derived for a particular consequence interpretation. The distinction matters because the behavioral summary can be

retained when the quantitative loss model changes, while the direct quantitative analysis can preserve control-flow distinctions that may be lost by revaluing an aggregate behavioral summary.

This perspective is closely related to abstract interpretation [8]. Indeed, type-and-effect systems and abstract interpretation are not competing approaches. Gordon's recent work establishes formal connections between effect quantales and abstract interpretations of event occurrences [21]. Our behavioral summaries can likewise be viewed as static abstractions of execution behavior. We choose a type-and-effect presentation because it exposes the compositional rules and their associated soundness guarantees directly. We do not establish a general abstract-interpretation construction or claim advantages in precision or expressiveness over alternative abstract interpreters.

Resource-aware and cost-aware type systems. Resource-aware type systems provide an extensive foundation for compositional quantitative reasoning. Automatic amortized resource analysis (AARA) derives symbolic bounds on program resource consumption using potential-based typing and sound operational cost semantics [26, 27]. Subsequent developments extend these techniques to nonlinear bounds, different resource metrics, and richer language constructs [28, 32]. AARA also supports user-defined cost metrics, so its applicability is not restricted to a single computational resource.

Liquid Resource Types further combine refinement typing with amortized resource analysis to verify resource bounds that depend on program values, including polynomial and exponential bounds [30]. These results establish that symbolic quantitative typing, value-dependent resource specifications, and compositional cost inference are well-developed techniques.

Our framework addresses a different semantic organization of quantitative properties. Rather than beginning solely with an operational cost metric, we distinguish hazardous behavior from the quantitative consequences assigned to that behavior. An execution first induces hazards, which are then interpreted through a domain-specific loss model. The static analysis separately exposes a hazard summary and a directly inferred quantitative bound.

This distinction supports the loss-parametric revaluation result of Theorem 2.5: once a sound hazard summary has been obtained, it can be interpreted under another monotone loss model without recomputing that behavioral abstraction.

Our assumption-guarded extension similarly has a different objective from the inference mechanisms developed in AARA or Liquid Resource Types. We do not attempt automatic inference of sophisticated value-dependent resource bounds. Instead, we propagate symbolic quantitative summaries and certify them under explicit assumptions about program parameters and deployment constraints. The approaches are complementary, and integrating more advanced quantitative inference techniques could improve the precision and automation of our system.

Sequential effects and history-sensitive analysis. Conventional effect systems often use commutative abstractions that do not retain execution order. Sequential effect systems address this limitation by representing effects whose composition depends on ordering.

Gordon develops an algebraic account of sequential effects using *effect quantales*, providing general composition and iteration mechanisms for order-sensitive effect systems [20]. This establishes a substantial foundation for reasoning about effect ordering and iteration independently of our work.

Earlier history-effect systems also provide important precedents. Bartoletti et al. develop a type-and-effect analysis that over-approximates resource-usage histories and checks whether those histories satisfy security and usage policies [5]. Their approach supports policies over event sequences and includes techniques for analyzing potentially unbounded history expressions.

Consequently, the use of an abstract history domain is not itself our contribution. Our history-sensitive extension builds on the general idea of compositional behavioral summaries but gives

those summaries a particular consequence-aware interpretation:

$$Q \xrightarrow{\eta} \mathcal{P}(W) \xrightarrow{L} X.$$

The abstract history domain Q represents execution information, η recognizes hazards induced by that information, and L assigns quantitative consequences to the recognized hazards.

This factorization supports a distinction between maintaining a behavioral history summary and interpreting that summary for a particular risk policy. In particular, Theorem 6.4 establishes that a soundly inferred history summary can be reinterpreted under alternative monotone hazard-extraction and loss functions without recomputing the behavioral abstraction.

The precision of such reinterpretation is limited by the information retained in Q. We do not claim that every temporal policy can be expressed using a given history domain, or that our abstract history construction is more expressive than existing history-effect systems.

Metric-aware secure service orchestration. The work most closely related to ours is the framework of Costa, Martinelli, and Yautsiukhin for metric-aware secure service orchestration [6]. Building on earlier history-effect techniques, they use a type-and-effect system to extract behavioral history expressions from service implementations. The history expressions incorporate security requirements and quantitative metric annotations, while c-semiring structures provide algebraic composition and verification of metric values against constraints.

Their subsequent work extends this approach to simultaneous reasoning about multiple security metrics [7]. These results already demonstrate that quantitative security requirements, multidimensional metrics, and history-based behavioral analysis can be combined within a type-and-effect framework.

The relationship is therefore substantial. Both approaches provide compositional static reasoning, associate quantitative information with security-relevant behavior, and support checking quantitative requirements before execution. Furthermore, their framework handles language constructs and service-composition mechanisms beyond the small sequential calculus used in our formal development.

Our formulation emphasizes a different separation of semantic responsibilities. In metric-aware service orchestration, quantitative metrics are associated with event and history expressions and combined according to the selected metric algebra. In our framework, behavioral abstraction, hazard recognition, and quantitative valuation are represented by distinct interfaces. In particular, the history summary q is not itself a quantitative metric, and the intermediate hazard interpretation $\eta(q)$ is independent of the subsequent loss assignment $L(\eta(q))$.

This organization permits a quantitative consequence to be associated with a compound hazardous condition recognized from a history, rather than requiring the entire consequence to be attributed locally to an individual operation. For example, the combination of public network exposure and customer-database attachment can induce an additional hazard whose loss is assigned only after the relevant behavioral information has been composed.

The corresponding reinterpretation theorem formalizes another consequence of this organization: the same sound history summary can support alternative admissible hazard interpretations and loss models.

We view this as a difference in semantic decomposition and the guarantees exposed by the analysis, not as a proof of greater expressive power. A systematic encoding or expressiveness comparison with metric-aware orchestration remains outside the scope of this work.

Quantitative information flow and security semantics. Quantitative information flow (QIF) provides a principled semantic foundation for measuring information leakage, including entropy-based

and gain-based formulations [2]. In particular, generalized gain functions make the quantitative evaluation of information leakage sensitive to the adversary's objectives and operational context [3].

This work is relevant because it demonstrates that quantitative interpretation need not be fixed by program behavior alone. Different gain functions can assign different values to the same probabilistic information channel. Thus, the general idea of separating behavioral semantics from application-dependent valuation has important precedents in security analysis.

Our approach considers a different class of semantic properties. QIF typically reasons about probabilistic information channels, observations, and relationships between executions. Our current framework instead assigns worst-case consequences to hazards recognized from individual execution histories or their abstractions.

We do not claim that arbitrary QIF measures can be represented by the present hazard-set semantics, nor that our framework subsumes probabilistic or relational security analyses. Rather, QIF illustrates the broader importance of interpreting program behavior through explicit, application-dependent quantitative models. Extending our framework to such probabilistic and relational properties would require additional semantic structure.

Quantitative verification and model checking. Quantitative verification analyzes numerical properties of transition systems, including accumulated rewards, probabilities of reaching undesirable states, expected costs, and multi-objective constraints [15, 16, 22]. Tools such as PRISM [31] and Storm [24] provide practical algorithms for analyzing probabilistic models.

These techniques offer richer stochastic semantics than those considered in our current framework. Our system does not compute probabilities or expected losses; its quantitative guarantees are conservative worst-case bounds over modeled execution behavior.

The principal methodological difference lies in the analysis interface. Quantitative model checking operates on transition-system or probabilistic models, often using symbolic state-space representations and specialized verification algorithms. Our analysis instead derives summaries syntax-directly from program structure and externally supplied primitive annotations, without requiring an explicit state-space construction for the program under analysis.

The two approaches are complementary. Quantitative verification could help establish or validate the primitive consequence summaries used by our analysis, while compositional static summaries may offer a lightweight certification mechanism when detailed system models are unavailable or unnecessary.

Workflow and pipeline analysis. Workflow languages provide structured representations of task-oriented computation, including sequencing, conditional execution, and repeated or parallel tasks. They are widely used in scientific computation, data processing, and deployment automation [12, 13, 41].

Existing workflow analyses address properties such as data-flow correctness [43], type compatibility [10], provenance [1], and scheduling or resource-related properties [34].

Our prototype uses a subset of the Workflow Description Language (WDL) [38] as a frontend for consequence-aware static analysis. Workflow tasks are associated with externally supplied behavioral and quantitative annotations, and the resulting intermediate representation is analyzed compositionally.

The prototype is intended to demonstrate the implementation of the formal analysis on workflow-facing syntax, not to establish superiority over existing workflow analyzers. It supports only a subset of WDL, and the present evaluation does not provide a comparative assessment of precision, scalability, or analysis effort.

Runtime verification and assurance. Runtime verification checks concrete execution traces against formal specifications, including temporal and quantitative properties [4, 14, 17]. Runtime assurance extends this perspective with monitoring and intervention mechanisms intended to maintain system safety [40].

Our framework addresses complementary questions. Runtime techniques reason about behavior as it occurs, while our static analysis conservatively summarizes possible modeled executions before deployment.

The separation between behavioral histories, hazard recognition, and loss interpretation also suggests a possible interface between static and dynamic reasoning. A runtime monitor could maintain a concrete or abstract history, interpret that history through a hazard extractor, and evaluate its consequences under the same loss model used for static certification.

Such an integration is not implemented here. Its development would require an explicit relationship between runtime observations, static summaries, and the assumptions underlying quantitative certification.

Summary and positioning. Prior work establishes sophisticated mechanisms for quantitative typing, value-dependent resource analysis, ordered and history-sensitive effects, metric-aware service orchestration, quantitative security semantics, and probabilistic verification. Our work draws on these foundations rather than claiming novelty for their individual mechanisms.

The contribution is a consequence-aware semantic organization in which behavioral summaries, hazard recognition, quantitative valuation, and policy certification remain distinct. The core analysis provides both a reusable hazard summary and a direct, potentially more precise quantitative bound. The history-sensitive extension enriches the behavioral summary while retaining a separate hazard interpretation.

The associated soundness and reinterpretation results formalize how these summaries can be used for quantitative certification and reused under alternative admissible interpretations. Our present results concern conservative, worst-case analysis of a small sequential language and its workflow-oriented implementation; they do not establish greater expressiveness or empirical superiority over the related approaches discussed above.

10 Conclusion

We presented a consequence-aware static analysis framework for enforcing quantitative risk policies. The framework separates execution behavior, hazard recognition, quantitative loss, and policy certification, providing distinct interfaces for describing what a program may do, which behaviors constitute hazards, and what consequences those hazards incur. We realize this architecture through a compositional type-and-effect system that computes conservative hazard summaries and quantitative risk bounds, with soundness guarantees relating both to execution semantics.

A central property of the framework is that behavioral abstraction and quantitative valuation need not be fixed together. In the core system, the hazard summary can be revalued under alternative monotone loss models without recomputing the hazard analysis. The history-sensitive extension generalizes this separation through an abstract history domain Q, a hazard interpretation η, and a loss model L:

$$Q \xrightarrow{\eta} \mathcal{P}(W) \xrightarrow{L} X.$$

This allows quantitative consequences to arise from hazards recognized only after behavioral histories have been composed. The corresponding reinterpretation result establishes conditions under which a sound history summary can be reused with alternative hazard interpretations and loss models. Meanwhile, the directly inferred quantitative effect provides a potentially more precise bound for a particular interpretation.

We extended the framework with prefix-based semantics for potentially non-terminating executions, fixed-point reasoning for unbounded iteration, and symbolic risk bounds that can be certified under explicit assumptions on program parameters. A prototype checker for a subset of the Workflow Description Language demonstrates how externally supplied primitive annotations, history-sensitive hazards, symbolic bounded repetition, and quantitative policies can be integrated into a working static analysis. The resulting guarantees are conditional on the soundness of the supplied annotations and the chosen hazard and loss models. Moreover, failure to establish a static bound within a policy threshold indicates that certification is unsuccessful, not necessarily that a concrete execution violates the policy.

Overall, this work contributes a semantic organization and sound static realization of consequence-aware reasoning, building on established techniques for quantitative analysis and effect tracking. By making behavioral summaries and their domain-specific interpretations explicit, the framework provides a foundation for developing static tools that certify quantitative consequences of execution and support reassessment when hazard definitions, loss models, or policy requirements change.

References

[1] Pinar Alper, Khalid Belhajjame, and Carole A Goble. 2017. Static analysis of Taverna workflows to predict provenance patterns. *Future Generation Computer Systems* 75 (2017), 310–329.

[2] Mário S Alvim, Konstantinos Chatzikokolakis, Annabelle McIver, Carroll Morgan, Catuscia Palamidessi, and Geoffrey Smith. 2020. *The science of quantitative information flow.* Springer.

[3] Mário S. Alvim, Konstantinos Chatzikokolakis, Catuscia Palamidessi, and Geoffrey Smith. 2012. Measuring Information Leakage Using Generalized Gain Functions. In *2012 IEEE 25th Computer Security Foundations Symposium*. IEEE, 265–279. doi:10.1109/CSF.2012.26

[4] Ezio Bartocci, Yliès Falcone, Adrian Francalanza, and Giles Reger. 2018. Introduction to runtime verification. In *Lectures on Runtime Verification: Introductory and Advanced Topics*. Springer, 1–33.

[5] Massimo Bartoletti, Pierpaolo Degano, Gian Luigi Ferrari, and Roberto Zunino. 2007. Types and Effects for Resource Usage Analysis. In *Foundations of Software Science and Computational Structures (Lecture Notes in Computer Science, Vol. 4423)*. Springer, 32–47. doi:10.1007/978-3-540-71389-0_4

[6] Gabriele Costa, Fabio Martinelli, and Artsiom Yautsiukhin. 2012. Metric-Aware Secure Service Orchestration. *Electronic Proceedings in Theoretical Computer Science* 104 (2012), 32–46. doi:10.4204/EPTCS.104.4

[7] Gabriele Costa, Fabio Martinelli, and Artsiom Yautsiukhin. 2014. Multi-dimensional Secure Service Orchestration. In *Business Process Management Workshops*. Springer, 473–485. doi:10.1007/978-3-319-06257-0_37

[8] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*. 238–252.

[9] Frank Cremer, Barry Sheehan, Michael Fortmann, Arash N Kia, Martin Mullins, Finbarr Murphy, and Stefan Materne. 2022. Cyber risk and cybersecurity: a systematic review of data availability. *The Geneva papers on risk and insurance. Issues and practice* 47, 3 (2022), 698.

[10] Vasa Curcin, Moustafa Ghanem, and Yike Guo. 2010. The design and implementation of a workflow analysis tool. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 368, 1926 (2010), 4193–4208.

[11] Francesco Dagnino, Paola Giannini, and Elena Zucca. 2025. Monadic type-and-effect soundness. *arXiv preprint arXiv:2504.10159* (2025).

[12] Stefano Dalla Palma, Dario Di Nucci, Fabio Palomba, and Damian Andrew Tamburri. 2020. Toward a catalog of software quality metrics for infrastructure code. *Journal of Systems and Software* 170 (2020), 110726.

[13] Paolo Di Tommaso, Maria Chatzou, Evan W Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow enables reproducible computational workflows. *Nature biotechnology* 35, 4 (2017), 316–319.

[14] Alexandre Donzé. 2013. On signal temporal logic. In *International Conference on Runtime Verification*. Springer, 382–383.

[15] Ingy Elsayed-Aly, David Parker, and Lu Feng. 2024. Distributional probabilistic model checking. In *NASA Formal Methods Symposium*. Springer, 57–75.

[16] Kousha Etessami, Marta Kwiatkowska, Moshe Y Vardi, and Mihalis Yannakakis. 2008. Multi-objective model checking of Markov decision processes. *Logical Methods in Computer Science* 4 (2008).

[17] Bernd Finkbeiner, Martin Fränzle, Florian Kohn, and Paul Kröger. 2022. A truly robust signal temporal logic: Monitoring safety properties of interacting cyber-physical systems under uncertain observation. *Algorithms* 15, 4 (2022), 126.

[18] Carlo A Furia, Bertrand Meyer, and Sergey Velder. 2014. Loop invariants: Analysis, classification, and examples. *ACM Computing Surveys (CSUR)* 46, 3 (2014), 1–51.

[19] Cunyuan Gao and Lionel Parreaux. 2025. A Lightweight Type-and-Effect System for Invalidation Safety: Tracking Permanent and Temporary Invalidation with Constraint-Based Subtype Inference. *Proceedings of the ACM on Programming Languages* 9, OOPSLA2 (2025), 2623–2653.

[20] Colin S Gordon. 2021. Polymorphic iterable sequential effect systems. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 43, 1 (2021), 1–79.

[21] Colin S. Gordon. 2026. Effect Systems as Abstract Interpretations. Also listed for the International Workshop on Numerical and Symbolic Abstract Domains (NSAD 2026). arXiv:2606.19686 [cs.PL] doi:10.48550/arXiv.2606.19686

[22] Arnd Hartmanns, Sebastian Junges, Tim Quatmann, and Maximilian Weininger. 2023. A practitioner's guide to MDP model checking algorithms. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 469–488.

[23] Julien Henry, David Monniaux, and Matthieu Moy. 2012. Pagai: A path sensitive static analyser. *Electronic Notes in Theoretical Computer Science* 289 (2012), 15–25.

[24] Christian Hensel, Sebastian Junges, Joost-Pieter Katoen, Tim Quatmann, and Matthias Volk. 2022. The probabilistic model checker Storm. *International Journal on Software Tools for Technology Transfer* 24, 4 (2022), 589–610.

[25] Daniel Hillerström, Sam Lindley, and Robert Atkey. 2020. Effect handlers via generalised continuations. *Journal of Functional Programming* 30 (2020), e5.

[26] Jan Hoffmann, Klaus Aehlig, and Martin Hofmann. 2012. Multivariate amortized resource analysis. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 34, 3 (2012), 1–62.

[27] Jan Hoffmann and Steffen Jost. 2022. Two decades of automatic amortized resource analysis. *Mathematical Structures in Computer Science* 32, 6 (2022), 729–759.

[28] David M Kahn and Jan Hoffmann. 2020. Exponential automatic amortized resource analysis. In *International Conference on Foundations of Software Science and Computation Structures*. Springer, 359–380.

[29] Samina Kanwal and Wan Fokkink. 2025. The impact of security threats on safety-critical systems: a systematic literature review and guidance for future research. *Available at SSRN 4945044* (2025).

[30] Tristan Knoth, Di Wang, Adam Reynolds, Jan Hoffmann, and Nadia Polikarpova. 2020. Liquid Resource Types. *Proceedings of the ACM on Programming Languages* 4, ICFP (2020), 106:1–106:29. doi:10.1145/3408988

[31] Marta Kwiatkowska, Gethin Norman, and David Parker. 2011. PRISM 4.0: Verification of probabilistic real-time systems. In *International conference on computer aided verification*. Springer, 585–591.

[32] Benjamin Lichtman and Jan Hoffmann. 2017. Arrays and references in resource aware ML. In *2nd International Conference on Formal Structures for Computation and Deduction (FSCD 2017)*. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 26–1.

[33] John M Lucassen and David K Gifford. 1988. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 47–57.

[34] Bertram Ludäscher, Ilkay Altintas, Chad Berkley, Dan Higgins, Efrat Jaeger, Matthew Jones, Edward A Lee, Jing Tao, and Yang Zhao. 2006. Scientific workflow management and the Kepler system. *Concurrency and computation: Practice and experience* 18, 10 (2006), 1039–1065.

[35] Daniel Marino and Todd Millstein. 2009. A generic type-and-effect system. In *Proceedings of the 4th international workshop on Types in language design and implementation*. 39–50.

[36] Jhonny Mertz and Ingrid Nunes. 2023. Software runtime monitoring with adaptive sampling rate to collect representative samples of execution traces. *Journal of Systems and Software* 202 (2023), 111708.

[37] Anders Møller and Michael I Schwartzbach. 2025. *Static program analysis*. Chapter Path sensitivity and relational analysis, 89–98.

[38] OpenWDL. [n. d.]. Workflow Description Language (WDL). https://openwdl.org/. Accessed: 2026-07-07.

[39] Akond Rahman and Laurie Williams. 2019. Source code properties of defective infrastructure as code scripts. *Information and Software Technology* 112 (2019), 148–163.

[40] Umberto Ravaioli, Kyle Dunlap, and Kerianne Hobbs. 2022. A universal framework for generalized run time assurance with JAX automatic differentiation. *arXiv preprint arXiv:2209.01120* (2022).

[41] Michael J Roach, N Tessa Pierce-Ward, Radoslaw Suchecki, Vijini Mallawaarachchi, Bhavya Papudeshi, Scott A Handley, C Titus Brown, Nathan S Watson-Haigh, and Robert A Edwards. 2022. Ten simple rules and a template for creating workflows-as-applications. *PLoS computational biology* 18, 12 (2022), e1010705.

[42] Benno Stein, Benjamin Barslev Nielsen, Bor-Yuh Evan Chang, and Anders Møller. 2019. Static analysis with demand-driven value refinement. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–29.

[43] Nikola Trčka, Wil MP Van der Aalst, and Natalia Sidorova. 2009. Data-flow anti-patterns: Discovering data-flow errors in workflows. In *International Conference on Advanced Information Systems Engineering*. Springer, 425–439.

[44] Orpheas van Rooij and Robbert Krebbers. 2025. Affect: An affine type and effect system. *Proceedings of the ACM on Programming Languages* 9, POPL (2025), 126–154.

A Proofs

Throughout this appendix, we use the algebraic assumptions of Sections 2 and 6. The quantitative composition operator $\oplus$ is associative and monotone, with identity $\mathbf{0}$, and $\mathbf{0}$ is the least quantitative element. The operator $\sqcup$ upper-bounds both its arguments.

For the history domain, q_0 is the least element, $\otimes$ is associative and monotone with identity q_0, and $\sqcup_Q$ upper-bounds both its arguments. Consequently, $r \preceq r \oplus r'$ and $q \preceq_Q q \otimes q'$.

A.1 Proof of Lemma 2.1

Let π_1 and π_2 be composable traces. By definition, $\mathsf{hazards}(\pi)$ is the union of the hazard labels emitted along π. The concatenation $\pi_1 \cdot \pi_2$ contains exactly the steps of π_1 followed by those of π_2. Therefore, $\mathsf{hazards}(\pi_1 \cdot \pi_2) = \mathsf{hazards}(\pi_1) \cup \mathsf{hazards}(\pi_2)$. □

A.2 Proof of Theorem 2.5

Suppose H is hazard sound for e, so $\Psi(e) \subseteq H$. Let L be any monotone loss model.

For every $\pi \in \mathsf{Traces}(e)$, we have $\mathsf{hazards}(\pi) \subseteq \Psi(e) \subseteq H$. Monotonicity of L gives $R_L(\pi) = L(\mathsf{hazards}(\pi)) \preceq L(H)$.

Thus $L(H)$ upper-bounds the risk of every terminating trace. Taking their least upper bound yields $R_L(e) = \bigsqcup_{\pi \in \mathsf{Traces}(e)} R_L(\pi) \preceq L(H)$.

The proof requires only hazard soundness and monotonicity of L; sequential compositionality is not needed. □

A.3 Proof of Lemma 3.2

We establish the three inclusions by considering an arbitrary terminating execution trace.

Sequencing. Let $\pi \in \mathsf{Traces}(e_1; e_2)$. By the operational semantics, π consists of a terminating execution π_1 of e_1, a pure sequencing reduction, and a terminating execution π_2 of e_2.

The sequencing reduction emits no hazards. By Lemma 2.1, $\mathsf{hazards}(\pi) = \mathsf{hazards}(\pi_1) \cup \mathsf{hazards}(\pi_2)$.

Since $\mathsf{hazards}(\pi_i) \subseteq \Psi(e_i)$, we obtain $\mathsf{hazards}(\pi) \subseteq \Psi(e_1) \cup \Psi(e_2)$. Taking the union over all such traces gives $\Psi(e_1; e_2) \subseteq \Psi(e_1) \cup \Psi(e_2)$.

Conditionals. Let π be a terminating trace of if e then e_1 else e_2. The trace evaluates the guard, performs a pure branch-selection step, and executes exactly one branch e_i, for $i \in \{1, 2\}$.

Writing π_g and π_i for the corresponding execution segments, Lemma 2.1 gives $\mathsf{hazards}(\pi) = \mathsf{hazards}(\pi_g) \cup \mathsf{hazards}(\pi_i)$.

Therefore, $\mathsf{hazards}(\pi) \subseteq \Psi(e) \cup \Psi(e_1) \cup \Psi(e_2)$. Taking the union over all conditional traces proves $\Psi(\text{if } e \text{ then } e_1 \text{ else } e_2) \subseteq \Psi(e) \cup \Psi(e_1) \cup \Psi(e_2)$.

Primitive application. Let $\pi \in \mathsf{Traces}(f(e))$. The trace evaluates the argument and then performs a primitive reduction emitting H_f. If π_e denotes the argument execution, $\mathsf{hazards}(\pi) = \mathsf{hazards}(\pi_e) \cup H_f$.

Since $\mathsf{hazards}(\pi_e) \subseteq \Psi(e)$, we have $\mathsf{hazards}(\pi) \subseteq \Psi(e) \cup H_f$. Taking the union over all traces yields $\Psi(f(e)) \subseteq \Psi(e) \cup H_f$. □

A.4 Proof of Theorem 3.3

We proceed by induction on the typing derivation $\Gamma \vdash e : \tau \triangleright (H, r)$.

Variables and values. Variables, unit, and boolean values emit no hazards. Therefore $\Psi(e) = \emptyset \subseteq H$.

Sequencing. Suppose the final rule is T-Seq, with premises $\Gamma \vdash e_i : \tau_i \triangleright (H_i, r_i)$, where $\tau_1 = \text{unit}$.
By induction, $\Psi(e_1) \subseteq H_1$ and $\Psi(e_2) \subseteq H_2$. Using Lemma 3.2, $\Psi(e_1; e_2) \subseteq \Psi(e_1) \cup \Psi(e_2) \subseteq H_1 \cup H_2$, which is the inferred hazard summary.

Conditionals. Suppose the final rule is T-If. Let H_g, H_1, H_2 denote the hazard summaries of the guard and branches.
The induction hypotheses give $\Psi(e_g) \subseteq H_g$, $\Psi(e_1) \subseteq H_1$, and $\Psi(e_2) \subseteq H_2$.
By Lemma 3.2, $\Psi(\text{if } e_g \text{ then } e_1 \text{ else } e_2) \subseteq \Psi(e_g) \cup \Psi(e_1) \cup \Psi(e_2) \subseteq H_g \cup H_1 \cup H_2$.

Primitive application. Suppose the final rule is T-Prim. The argument has hazard summary H and the primitive has annotation H_f.
By induction, $\Psi(e) \subseteq H$. Lemma 3.2 gives $\Psi(f(e)) \subseteq \Psi(e) \cup H_f \subseteq H \cup H_f$.
Thus every core typing rule preserves hazard soundness. □

A.5 Proof of Theorem 3.5

Fix a loss model L satisfying Assumption 2.2, and assume that primitive annotations satisfy Assumption 3.1.
We prove that $\Gamma \vdash e : \tau \triangleright (H, r)$ implies $R_L(e) \preceq r$, by induction on the typing derivation.

Variables and values. These expressions emit no hazards. Since $L(\emptyset) = \mathbf{0}$, $R_L(e) = \mathbf{0} \preceq r$.

Sequencing. Suppose the final rule is T-Seq, with inferred risk $r_1 \oplus r_2$.
Let $\pi \in \text{Traces}(e_1; e_2)$. Its hazards arise from traces π_1 and π_2 of the two subexpressions, so $\text{hazards}(\pi) = \text{hazards}(\pi_1) \cup \text{hazards}(\pi_2)$.
Sequential compositionality gives $R_L(\pi) \preceq L(\text{hazards}(\pi_1)) \oplus L(\text{hazards}(\pi_2))$.
By induction, $L(\text{hazards}(\pi_i)) \preceq r_i$. Monotonicity of $\oplus$ therefore yields $R_L(\pi) \preceq r_1 \oplus r_2$.
Taking the supremum over all terminating traces gives $R_L(e_1; e_2) \preceq r_1 \oplus r_2$.

Conditionals. Suppose the final rule is T-If, with inferred risk $r_g \oplus (r_1 \sqcup r_2)$.
Every terminating trace evaluates the guard followed by exactly one branch e_i. For the corresponding traces π_g, π_i, sequential compositionality gives $R_L(\pi) \preceq L(\text{hazards}(\pi_g)) \oplus L(\text{hazards}(\pi_i))$.
By induction and monotonicity, $R_L(\pi) \preceq r_g \oplus r_i$. Since $r_i \preceq r_1 \sqcup r_2$, we have $R_L(\pi) \preceq r_g \oplus (r_1 \sqcup r_2)$.
Taking the supremum over all traces establishes the desired bound.

Primitive application. Suppose the final rule is T-Prim, with inferred risk $r \oplus r_f$.
A terminating trace consists of an argument execution π_e followed by a primitive step emitting H_f. Sequential compositionality gives $R_L(\pi) \preceq L(\text{hazards}(\pi_e)) \oplus L(H_f)$.
By induction, $L(\text{hazards}(\pi_e)) \preceq r$. Primitive quantitative soundness gives $L(H_f) \preceq r_f$.
Hence $R_L(\pi) \preceq r \oplus r_f$. Taking the supremum over terminating traces completes the proof. □

A.6 Proof of Lemma 4.1

We strengthen the core soundness argument to finite execution prefixes. The induction establishes both $\text{hazards}(\sigma) \subseteq H$ and $L(\text{hazards}(\sigma)) \preceq r$ for every prefix σ of a well-typed expression.
The argument proceeds by structural induction on the typing derivation. For subexpressions containing iteration, we use their corresponding prefix-soundness properties, established simultaneously by the iteration cases below.

Variables and values. Every prefix emits no hazards and incurs zero loss. Both conclusions follow immediately.

Sequencing. A prefix of $e_1; e_2$ either stops during e_1 or reaches e_2.

In the first case, the induction hypothesis gives $\mathsf{hazards}(\sigma) \subseteq H_1$ and $L(\mathsf{hazards}(\sigma)) \preceq r_1$. Since $H_1 \subseteq H_1 \cup H_2$ and $r_1 = r_1 \oplus \mathbf{0} \preceq r_1 \oplus r_2$, the conclusion follows.

In the second case, the prefix consists of a complete execution of e_1, a pure sequencing reduction, and a prefix of e_2. The induction hypotheses and sequential compositionality of L give the bounds $H_1 \cup H_2$ and $r_1 \oplus r_2$.

Conditionals. A prefix either remains within guard evaluation or enters one branch.

A guard-only prefix is bounded by H_g and r_g. These are below the inferred summaries $H_g \cup H_1 \cup H_2$ and $r_g \oplus (r_1 \sqcup r_2)$.

A prefix entering branch i is bounded by $H_g \cup H_i$ and $r_g \oplus r_i$. Since $r_i \preceq r_1 \sqcup r_2$, these are again below the inferred summaries.

Primitive application. A prefix either remains within argument evaluation or includes the primitive step.

The former is bounded by the argument summary (H, r), which is below $(H \cup H_f, r \oplus r_f)$. In the latter case, the primitive emits H_f, and its loss is bounded by r_f. Sequential compositionality yields the required bound.

Thus the core rules preserve hazard and risk soundness for all finite prefixes. □

A.7 Proof of Lemma 4.2

Suppose H is prefix-hazard sound for e, so $\Psi(e) \subseteq H$. Let L' be any monotone loss model.

For every $\sigma \in \mathsf{Prefixes}(e)$, $\mathsf{hazards}(\sigma) \subseteq \Psi(e) \subseteq H$. Monotonicity gives $R_{L'}(\sigma) = L'(\mathsf{hazards}(\sigma)) \preceq L'(H)$.

Taking the supremum over all finite prefixes, $R_{L'}(e) \preceq L'(H)$. □

A.8 Proof of Lemma 4.3

Let $\sigma \in \mathsf{Prefixes}(\text{while } e \text{ do } b)$.

Every finite prefix contains finitely many loop unfoldings. Each unfolding uses E-While and emits $\emptyset$. All remaining effectful steps belong to evaluations of the guard e or body b.

Every such segment is a complete execution or a finite prefix of the corresponding expression. Consequently, every hazard emitted by σ belongs to $\Psi(e)$ or $\Psi(b)$.

Thus $\mathsf{hazards}(\sigma) \subseteq \Psi(e) \cup \Psi(b)$. Taking the union over all loop prefixes yields $\Psi(\text{while } e \text{ do } b) \subseteq \Psi(e) \cup \Psi(b)$. □

A.9 Proof of Lemma 4.4

We proceed by induction on $n \in \mathbb{N}$.

For $n = 0$, the expression repeat 0 do b reduces to $()$ without emitting hazards. Hence its semantic hazard set is empty.

Suppose the result holds for $n = k$. The expression repeat $(k+1)$ do b reduces to b; repeat k do b through a pure reduction.

Every hazard emitted by a prefix of the resulting computation arises from a prefix of b or from a prefix of repeat k do b. By the induction hypothesis, both are contained in $\Psi(b)$.

Therefore $\Psi(\text{repeat } (k+1) \text{ do } b) \subseteq \Psi(b)$. □

A.10 Proof of Theorem 4.5

We extend the hazard-soundness induction of Theorem 3.3 with the two iteration rules.

Unbounded iteration. Suppose T-While derives $\Gamma \vdash \text{while } e \text{ do } b : \text{unit} \triangleright (H_g \cup H_b, r_\ell)$.

The induction hypotheses give $\Psi(e) \subseteq H_g$ and $\Psi(b) \subseteq H_b$. Using Lemma 4.3, $\Psi(\text{while } e \text{ do } b) \subseteq \Psi(e) \cup \Psi(b) \subseteq H_g \cup H_b$.

Bounded repetition. Suppose T-Repeat derives $\Gamma \vdash \text{repeat } n \text{ do } b : \text{unit} \triangleright (H_b, n \cdot r_b)$.
By induction, $\Psi(b) \subseteq H_b$. Lemma 4.4 gives $\Psi(\text{repeat } n \text{ do } b) \subseteq \Psi(b) \subseteq H_b$.
Both iterative constructs preserve hazard soundness. □

A.11 Proof of Lemma 4.7

The concatenation $\sigma_1 \cdot \sigma_2$ contains exactly the steps of σ_1 followed by those of σ_2. Since hazards accumulate by union, $\mathsf{hazards}(\sigma_1 \cdot \sigma_2) = \mathsf{hazards}(\sigma_1) \cup \mathsf{hazards}(\sigma_2)$. □

A.12 Proof of Theorem 4.8

We extend the prefix-soundness argument to the two iteration constructs.

Unbounded iteration. Suppose T-While derives $\Gamma \vdash \text{while } e \text{ do } b : \text{unit} \triangleright (H_g \cup H_b, r_\ell)$, with guard and body risk bounds r_g, r_b and closure premise $r_g \sqcup (r_g \oplus r_b \oplus r_\ell) \preceq r_\ell$.
We show that every finite loop prefix σ satisfies $L(\mathsf{hazards}(\sigma)) \preceq r_\ell$.
The proof proceeds by induction on the number of complete iterations followed by further loop execution within σ.
First suppose σ stops during the guard, after a false guard, or during the first body execution. Prefix soundness of the guard and body bounds its loss by r_g or $r_g \oplus r_b$, respectively.
The closure premise implies $r_g \preceq r_\ell$ and $r_g \oplus r_b \oplus r_\ell \preceq r_\ell$. Since $\mathbf{0} \preceq r_\ell$ and $\oplus$ is monotone with identity $\mathbf{0}$, $r_g \oplus r_b \preceq r_\ell$. Thus these prefixes satisfy the required bound.
Now suppose σ contains a completed iteration followed by another loop prefix. Ignoring pure reductions, it decomposes into a completed guard execution σ_g, a completed body execution σ_b, and a strictly shorter loop prefix σ'.
By Lemma 4.7 and sequential compositionality, $L(\mathsf{hazards}(\sigma)) \preceq L(\mathsf{hazards}(\sigma_g)) \oplus L(\mathsf{hazards}(\sigma_b)) \oplus L(\mathsf{hazards}(\sigma'))$.
Prefix soundness of the guard and body, together with the induction hypothesis for σ', gives $L(\mathsf{hazards}(\sigma)) \preceq r_g \oplus r_b \oplus r_\ell \preceq r_\ell$.
Thus every finite prefix is bounded by r_ℓ. Taking the supremum yields $R_L(\text{while } e \text{ do } b) \preceq r_\ell$.

Bounded repetition. Suppose T-Repeat derives $\Gamma \vdash \text{repeat } n \text{ do } b : \text{unit} \triangleright (H_b, n \cdot r_b)$.
We proceed by induction on n, using $0 \cdot r_b = \mathbf{0}$ and $(k+1) \cdot r_b = r_b \oplus (k \cdot r_b)$.
For $n = 0$, every prefix emits no hazards, so $R_L(\text{repeat } 0 \text{ do } b) = \mathbf{0} = 0 \cdot r_b$.
Suppose the result holds for $n = k$. A prefix of repeat $(k+1)$ do b either stops during its first body execution or reaches the remaining repetition.
In the first case, prefix soundness gives $L(\mathsf{hazards}(\sigma)) \preceq r_b$. Since $r_b \preceq r_b \oplus (k \cdot r_b)$, the required bound follows.
In the second case, the effectful portion decomposes into a complete execution σ_b of b followed by a prefix σ_k of repeat k do b.
Sequential compositionality gives $L(\mathsf{hazards}(\sigma)) \preceq L(\mathsf{hazards}(\sigma_b)) \oplus L(\mathsf{hazards}(\sigma_k))$.
By prefix soundness of b and the induction hypothesis, $L(\mathsf{hazards}(\sigma)) \preceq r_b \oplus (k \cdot r_b) = (k+1) \cdot r_b$.
Taking the supremum over all prefixes establishes $R_L(\text{repeat } (k+1) \text{ do } b) \preceq (k+1) \cdot r_b$.
Therefore both forms of iteration preserve direct quantitative risk soundness. □

A.13 Proof of Lemma 5.2

Suppose $\Phi; \Gamma \vdash e : \tau \triangleright (H, r)$ and let $\nu \models \Phi$ be an admissible valuation.
We prove $\nu(\Gamma) \vdash \nu(e) : \tau \triangleright (H, \nu(r))$ by induction on the typing derivation.

Variables and values. Valuation leaves ordinary variables, unit values, and boolean values unchanged. It also preserves $\emptyset$ and $\mathbf{0}$. Thus T-Var, T-Unit, and T-Bool remain applicable.

Sequencing. Suppose the final rule is T-Seq. By induction, the premises instantiate to $\nu(\Gamma) \vdash \nu(e_i) : \tau_i \triangleright (H_i, \nu(r_i))$.

Applying T-Seq gives hazard summary $H_1 \cup H_2$ and risk $\nu(r_1) \oplus \nu(r_2)$. Since valuation commutes with $\oplus$, this equals $\nu(r_1 \oplus r_2)$. The expression structure is preserved because $\nu(e_1; e_2) = \nu(e_1); \nu(e_2)$.

Conditionals. Suppose the final rule is T-If. By induction, valuation preserves the typing judgments of the guard and both branches.

Applying T-If gives quantitative bound $\nu(r_g) \oplus (\nu(r_1) \sqcup \nu(r_2))$. Because valuation commutes with $\oplus$ and $\sqcup$, this equals $\nu(r_g \oplus (r_1 \sqcup r_2))$. The hazard summary remains $H_g \cup H_1 \cup H_2$.

Primitive application. Suppose the final rule is T-Prim, with annotation $f : \tau_1 \xrightarrow{(H_f, r_f)} \tau_2 \in \Gamma$.

By induction, the argument judgment instantiates with hazard summary H and risk $\nu(r)$. By the definition of valuation on contexts, $f : \tau_1 \xrightarrow{(H_f, \nu(r_f))} \tau_2 \in \nu(\Gamma)$.

Applying T-Prim gives risk $\nu(r) \oplus \nu(r_f) = \nu(r \oplus r_f)$ and unchanged hazard summary $H \cup H_f$.

Unbounded iteration. Suppose the final rule is T-While, with closure premise $r_g \sqcup (r_g \oplus r_b \oplus r_\ell) \preceq r_\ell$.

The induction hypotheses preserve the guard and body judgments. Since valuation is monotone and commutes with $\oplus$ and $\sqcup$, the side condition becomes $\nu(r_g) \sqcup (\nu(r_g) \oplus \nu(r_b) \oplus \nu(r_\ell)) \preceq \nu(r_\ell)$.

Thus T-While remains applicable and derives the instantiated loop judgment with hazard summary $H_g \cup H_b$ and risk $\nu(r_\ell)$.

Bounded repetition. Suppose the final rule is T-Repeat-Sym, with body summary (H_b, r_b).

By induction, the instantiated body has summary $(H_b, \nu(r_b))$. Because ν is admissible and n occurs as a repetition bound, $\nu(n) \in \mathbb{N}$.

The concrete rule T-Repeat therefore derives the bound $\nu(n) \cdot \nu(r_b)$. By compatibility of valuation with scalar multiplication, this equals $\nu(n \cdot r_b)$.

Hence every typing rule is preserved under admissible valuation. □

A.14 Proof of Lemma 5.3

Suppose Γ is Φ-sound with respect to L and $\nu \models \Phi$.

For any primitive annotation $f : \tau_1 \xrightarrow{(H_f, r_f)} \tau_2 \in \Gamma$, Definition 5.1 gives $L(H_f) \preceq \nu(r_f)$.

The corresponding annotation in $\nu(\Gamma)$ is $f : \tau_1 \xrightarrow{(H_f, \nu(r_f))} \tau_2$. It therefore satisfies Assumption 3.1.

Since the primitive was arbitrary, every annotation in $\nu(\Gamma)$ is quantitatively sound with respect to L. □

A.15 Proof of Theorem 5.5

Suppose $\Phi; \Gamma \vdash e : \tau \triangleright (H, r)$, Γ is Φ-sound with respect to L, and $\Phi \models r \preceq \rho$.

Let $\nu \models \Phi$ be arbitrary. By Lemma 5.2, $\nu(\Gamma) \vdash \nu(e) : \tau \triangleright (H, \nu(r))$.

By Lemma 5.3, the instantiated primitive annotations satisfy the concrete primitive-soundness assumption.

The direct risk-soundness results for the core language and iteration therefore give $R_L(\nu(e)) \preceq \nu(r)$.

Since $\Phi \models r \preceq \rho$ and $\nu \models \Phi$, we have $\nu(r) \preceq \rho$. By transitivity, $R_L(\nu(e)) \preceq \nu(r) \preceq \rho$.

Because ν was arbitrary, the inequality holds for every admissible valuation satisfying Φ. Thus $e \models_{L,\Phi} \rho$. □

A.16 Proof of Lemma 6.1

Let σ_1 have step sequence $s_0, \ldots, s_{m-1}$ and σ_2 have step sequence $t_0, \ldots, t_{n-1}$.

By definition, $q_{\sigma_1} = \kappa(s_0) \otimes \cdots \otimes \kappa(s_{m-1})$ and $q_{\sigma_2} = \kappa(t_0) \otimes \cdots \otimes \kappa(t_{n-1})$.

Their concatenation has step sequence $s_0, \ldots, s_{m-1}, t_0, \ldots, t_{n-1}$. Associativity of $\otimes$ therefore gives $q_{\sigma_1 \cdot \sigma_2} = q_{\sigma_1} \otimes q_{\sigma_2}$.

The identity property of q_0 covers cases where either prefix is empty. □

A.17 Proof of Lemma 6.2

We prove that $\Gamma \vdash e : \tau \triangleright (q, r)$ implies $q_\sigma \preceq_Q q$ for every $\sigma \in \mathrm{Prefixes}(e)$.

The proof proceeds by induction on the typing derivation. Only the history component is relevant.

Variables and values. Variables, unit, and boolean values execute no history-contributing primitives. Every prefix therefore has summary q_0, which equals the inferred history effect.

Sequencing. Suppose H-Seq derives $\Gamma \vdash e_1; e_2 : \tau_2 \triangleright (q_1 \otimes q_2, r)$.

Let $\sigma \in \mathrm{Prefixes}(e_1; e_2)$.

If σ stops during evaluation of e_1, the induction hypothesis gives $q_\sigma \preceq_Q q_1$. Since q_0 is least and is the identity of $\otimes$, $q_1 = q_1 \otimes q_0 \preceq_Q q_1 \otimes q_2$. Hence $q_\sigma \preceq_Q q_1 \otimes q_2$.

Otherwise, σ extends past completion of e_1. Ignoring pure sequencing reductions, it consists of a complete execution σ_1 of e_1 followed by a prefix σ_2 of e_2.

By Lemma 6.1, $q_\sigma = q_{\sigma_1} \otimes q_{\sigma_2}$. The induction hypotheses give $q_{\sigma_1} \preceq_Q q_1$ and $q_{\sigma_2} \preceq_Q q_2$. Monotonicity of $\otimes$ yields $q_\sigma \preceq_Q q_1 \otimes q_2$.

Conditionals. Suppose H-If derives history summary $q_g \otimes (q_1 \sqcup_Q q_2)$.

If σ stops during guard evaluation, induction gives $q_\sigma \preceq_Q q_g$. Since $q_0 \preceq_Q q_1 \sqcup_Q q_2$, we obtain $q_g = q_g \otimes q_0 \preceq_Q q_g \otimes (q_1 \sqcup_Q q_2)$.

Otherwise, σ completes guard evaluation and enters one branch e_i. Ignoring pure reductions, $q_\sigma = q_{\sigma_g} \otimes q_{\sigma_i}$.

By induction, $q_{\sigma_g} \preceq_Q q_g$ and $q_{\sigma_i} \preceq_Q q_i$. Since $q_i \preceq_Q q_1 \sqcup_Q q_2$, monotonicity of $\otimes$ gives $q_\sigma \preceq_Q q_g \otimes (q_1 \sqcup_Q q_2)$.

Primitive application. Suppose H-Prim derives $\Gamma \vdash f(e) : \tau_2 \triangleright (q_e \otimes q_f, r)$.

If σ stops during argument evaluation, induction gives $q_\sigma \preceq_Q q_e$. Since $q_0 \preceq_Q q_f$, $q_e = q_e \otimes q_0 \preceq_Q q_e \otimes q_f$.

Otherwise, σ completes argument evaluation and includes the primitive step. By Lemma 6.1 and the definition of κ, $q_\sigma = q_{\sigma_e} \otimes q_f$. Induction gives $q_{\sigma_e} \preceq_Q q_e$, so $q_\sigma \preceq_Q q_e \otimes q_f$.

Thus every history-aware typing rule yields an upper bound on the abstract runtime histories of all execution prefixes. □

A.18 Proof of Theorem 6.3

Suppose $\Gamma \vdash e : \tau \triangleright (q, r)$, and let $\sigma \in \mathrm{Prefixes}(e)$ be arbitrary.

By Lemma 6.2, $q_\sigma \preceq_Q q$. Monotonicity of η gives $\eta(q_\sigma) \subseteq \eta(q)$.

Taking the union over all finite prefixes, $\bigcup_{\sigma \in \mathrm{Prefixes}(e)} \eta(q_\sigma) \subseteq \eta(q)$.

Hence the inferred history summary is hazard sound under interpretation η. □

A.19 Proof of Theorem 6.4

Suppose $\Gamma \vdash e : \tau \triangleright (q, r)$. Let $\eta' : Q \to \mathcal{P}(W)$ and $L' : \mathcal{P}(W) \to X$ be monotone interpretations satisfying the conditions of the theorem.

For every execution prefix $\sigma \in \mathrm{Prefixes}(e)$, Lemma 6.2 gives $q_\sigma \preceq_Q q$.

Monotonicity of η' implies $\eta'(q_\sigma) \subseteq \eta'(q)$. Monotonicity of L' then gives $L'(\eta'(q_\sigma)) \preceq L'(\eta'(q))$.

Thus $L'(\eta'(q))$ upper-bounds the consequence of every finite prefix. Taking the supremum yields $R_{\eta',L'}(e) = \bigsqcup_{\sigma \in \mathsf{Prefixes}(e)} L'(\eta'(q_\sigma)) \preceq L'(\eta'(q))$.

The result requires only history soundness and monotonicity of the interpretations. It does not depend on soundness of the original quantitative component r under η' or L'. □

A.20 Proof of Lemma 6.5

We proceed by case analysis on the final history-aware typing rule.

Variables and values. Rules H-Var, H-Unit, and H-Bool assign $(q, r) = (q_0, \mathbf{0})$. Since $\eta(q_0) = \emptyset$ and $L(\emptyset) = \mathbf{0}$, we have $L(\eta(q_0)) = \mathbf{0} \preceq r$.

Sequencing. Rule H-Seq infers $q = q_1 \otimes q_2$ and $r = (r_1 \oplus r_2) \sqcup L(\eta(q))$. Since $\sqcup$ upper-bounds both operands, $L(\eta(q)) \preceq r$.

Conditionals. Rule H-If infers $q = q_g \otimes (q_1 \sqcup_Q q_2)$ and $r = (r_g \oplus (r_1 \sqcup r_2)) \sqcup L(\eta(q))$. Again, $L(\eta(q)) \preceq r$.

Primitive application. Rule H-Prim infers $q = q_e \otimes q_f$ and $r = (r_e \oplus r_f) \sqcup L(\eta(q))$. The upper-bound property of $\sqcup$ gives $L(\eta(q)) \preceq r$.

Thus the consistency condition holds for every history-aware typing judgment. □

A.21 Proof of Theorem 6.6

Suppose $\Gamma \vdash e : \tau \triangleright (q, r)$, and let $\sigma \in \mathsf{Prefixes}(e)$ be arbitrary.

By Lemma 6.2, $q_\sigma \preceq_Q q$. Monotonicity of η gives $\eta(q_\sigma) \subseteq \eta(q)$.

By monotonicity of L, $L(\eta(q_\sigma)) \preceq L(\eta(q))$.

Lemma 6.5 establishes $L(\eta(q)) \preceq r$. Therefore $L(\eta(q_\sigma)) \preceq r$ for every execution prefix σ.

Taking the supremum over all prefixes, $R_{\eta,L}(e) = \bigsqcup_{\sigma \in \mathsf{Prefixes}(e)} L(\eta(q_\sigma)) \preceq r$.

Hence the history-aware type-and-effect system is directly risk sound under the chosen interpretations η and L. □